

How PostgreSQL Replication Actually Works, and How to Decide When a Service Deserves Its Own Boundary

A technical reference on the Write-Ahead Log, replication topology, query and concurrency design, and the service-extraction decision, built around the Tadka case study

Contents

How to read this document

Module 1: PostgreSQL Replication and the Write-Ahead Log

- 1.1 The crash-recovery problem the Write-Ahead Log solves
- 1.2 Inside the WAL: log sequence numbers, segments, and checkpoints
- 1.3 From crash recovery to streaming replication
- 1.4 Primary and replica roles
- 1.5 Replication lag: why a replica is never guaranteed to be caught up
- 1.6 Synchronous versus asynchronous replication: the durability-latency trade-off
- 1.7 Replication slots: how the primary knows what a replica still needs
- 1.8 Physical replication versus logical replication
- 1.9 Hot standby and the cost of read queries on a replica
- 1.10 Scaling to many replicas: direct fan-out versus cascading replication
- 1.11 Cascading replication in PostgreSQL: how it actually works
- 1.12 Failover and promotion: replication is not automatic failover
- 1.13 Split-brain: the risk a promotion tool exists specifically to prevent
- 1.14 Disaster recovery: point-in-time recovery and WAL archiving
- 1.15 Managed PostgreSQL: what the platform gives you, and what it still doesn't
- 1.16 Applying this to Tadka
- 1.17 Extended scenario bank

Module 2: Read Replicas in Application Architecture

- 2.1 Why add a read replica at all

2.2 Read-after-write consistency: the problem replicas create

2.3 Strategies for consistency-sensitive reads

2.4 Scale the bottleneck, not the symptom

2.5 Extended scenario bank

Module 3: The Service Extraction Decision Framework

3.1 The question that should never be answered with "because microservices"

3.2 Testing the weak reasons

3.3 Testing the legitimate reasons

3.4 The four-question test

3.5 Why yesterday's async decision makes today's extraction safer

3.6 Extended scenario bank

Module 4: Designing the Boundary

4.1 What actually moves: ownership, not just code

4.2 The interface-first principle, and its cross-stack equivalents

4.3 Two resilience boundaries and the timeout-budget problem

4.4 Database-per-service, and why a shared database breaks the boundary

4.5 Extended scenario bank

Module 5: Proving Fault Isolation

5.1 A claim is not a proof

5.2 Establishing the baseline

5.3 Killing the dependency: what should, and should not, fail

5.4 The architecture review checklist: five common mistakes

5.5 Extended scenario bank

Module 6: The Durability Wound: Space Versus Time

6.1 What extraction did, and did not, solve

6.2 Why RAM is not enough

6.3 Three options for making the work durable

6.4 Idempotency returns, and why it must

6.5 The architecture progression

6.6 Extended scenario bank

Module 7: Query and Schema Design at Scale

- 7.1 Indexing: workload-specific, not universal
- 7.2 Pagination: offset versus keyset
- 7.3 Identifier design: UUID, UUIDv7, and sequences
- 7.4 Extended scenario bank

Module 8: Concurrency Control

- 8.1 MVCC and the lost-update problem
- 8.2 Optimistic versus pessimistic locking
- 8.3 Deadlocks and lock ordering
- 8.4 Extended scenario bank

Module 9: Caching, Sharding, and the Payment Resilience Recap

- 9.1 Cache-aside and cache stampede
- 9.2 Cache invalidation strategy
- 9.3 Sharding strategies and the hot shard problem
- 9.4 Payment resilience recap
- 9.5 Extended scenario bank

Module 10: The Architect's Meta-Framework

- 10.1 The seven questions to ask before designing anything
- 10.2 The skill-progression framework
- 10.3 The one sentence worth internalizing

Appendix A: The System Design Question Bank, Developer to Architect

- A.1 Database indexing
- A.2 Offset, cursor, and keyset pagination
- A.3 UUIDs and index locality
- A.4 MVCC and concurrent updates
- A.5 Deadlocks
- A.6 Idempotency
- A.7 Read replicas
- A.8 Read replica consistency
- A.9 Cascading replication
- A.10 Caching
- A.11 Cache stampede

- A.12 Cache invalidation
- A.13 Sharding
- A.14 Hot shards
- A.15 Payment timeout
- A.16 Payment unknown outcome
- A.17 Bulkhead
- A.18 Circuit breaker
- A.19 Asynchronous processing
- A.20 Service extraction
- A.21 Database-per-service
- A.22 HTTP versus a message broker
- A.23 Enumerating failure across a network boundary
- A.24 The large-scale Tadka system design scenario

Appendix B: The Architect Defense Round

Appendix C: Configuration and Cross-Stack Reference

- C.1 Setting up asynchronous streaming replication
- C.2 Synchronous replication
- C.3 Cascading replication
- C.4 Monitoring replication lag
- C.5 Read/write connection routing per stack
- C.6 Idempotency key and durable payment job schemas

Appendix D: Glossary

Appendix E: Further Reading

How to read this document

This is not a PostgreSQL command reference or a checklist for standing up a replica. It is a reference on **architectural reasoning**, the same discipline this series has applied to Redis and to resilience patterns: classify the mechanism correctly, enumerate the real options, name what each option costs, and identify the new failure mode it introduces.

Every significant decision is presented as an **Architect's Decision block**:

Options: *what could we do*
Choice: *what we picked for Tadka*
Why: *the reasoning*
Trade-off: *what we gave up*
Failure mode: *how this specific choice breaks*
Revisit when: *the trigger that should make you reconsider*

A second recurring device in this document is the **Question Bank entry**: a realistic question, a plausible but incomplete "developer answer," the deeper "architect thinking" that follows, and the general principle it distills. These are drawn directly from a structured interview-and-defense exercise, rewritten here in full sentences so they read as a study reference rather than spoken transcript fragments.

Read every block with the same rule this series has repeated from the start: **a technology name is never the answer**. "We'll add a replica," "we'll shard by customer ID," "we'll extract Payment into a microservice" are not architectural decisions. They are implementation details of a decision that has not yet been stated out loud, in terms of the actual problem being solved.

Module 1

PostgreSQL Replication and the Write-Ahead Log

1.1 The crash-recovery problem the Write-Ahead Log solves

Start with the simplest possible mental model of a database: an application sends `UPDATE orders SET status = 'PAID' WHERE id = 101;`, and PostgreSQL changes the row. That model is complete for understanding *what* a database does, and dangerously incomplete for understanding *how it survives failure*.

Suppose PostgreSQL applies the change to its in-memory representation of the `orders` table, and the server crashes in the same instant, before that change is guaranteed to be durably on disk. On restart, how does PostgreSQL know the update ever happened? This is not a hypothetical edge case; it is the fundamental reliability problem every durable database must solve, and PostgreSQL's answer is the **Write-Ahead Log (WAL)**.

The rule the name itself encodes: **a change must be durably recorded in the log before the corresponding change to the actual data page is considered complete**. Concretely, the effect of every `INSERT`, `UPDATE`, and `DELETE` is first written as a WAL record, and only that WAL record needs to reach durable storage for the transaction to be considered committed. The data pages themselves (the actual table and index blocks in PostgreSQL's shared buffers) can be flushed to disk later, in the background, at PostgreSQL's convenience, because the WAL record is now the source of truth for "what happened, in what order," and can be replayed to reconstruct any data-page change that was lost when the crash occurred.

1.2 Inside the WAL: log sequence numbers, segments, and checkpoints

Treating the WAL as an abstract "ordered stream of changes" is a good starting mental model; a working architect needs the next layer of precision, because these specifics are exactly what later sections (replication lag, replication slots, point-in-time recovery) are built on.

The Log Sequence Number (LSN). Every position in the WAL stream has a unique, monotonically increasing address called an LSN, displayed as a hexadecimal pair like `0/1634F850`. An LSN is not a timestamp; it is a byte offset into the logical WAL stream, and it is the unit every replication and recovery mechanism in PostgreSQL uses to answer "how far has this WAL consumer gotten."

WAL segments. The WAL stream is physically stored as a sequence of fixed-size files called segments, 16MB each by default (configurable at cluster initialization time). PostgreSQL fills one segment, then rolls to the next, continuously, for as long as the server runs. This is why WAL retention and disk space are a real operational concern: every segment that some consumer (crash recovery, a replica, an archiving process) might still need has to be kept on disk, and every segment that no consumer needs can, and should, be recycled.

Checkpoints. Replaying every WAL record ever written, from the beginning of time, would make crash recovery take longer with every day the server has been running. PostgreSQL avoids this with **checkpoints**: a periodic operation (governed by `checkpoint_timeout`, five minutes by default, and `max_wal_size`, which forces an earlier checkpoint if too much WAL has accumulated) that guarantees every data-page change up to a given LSN has actually been flushed to disk. Crash recovery then only

needs to replay WAL from the most recent checkpoint forward, not from the beginning of the log, which is what keeps recovery time bounded regardless of how long the server has been running.

1.3 From crash recovery to streaming replication

The insight that turns a purely local durability mechanism into a distributed-systems tool is simple to state and easy to underestimate: if the WAL is a complete, ordered record of every change made to the database, there is no reason that stream has to stay on one machine. PostgreSQL can ship that same stream to a second PostgreSQL server, which replays it exactly the way crash recovery replays it locally, arriving at the same database state, continuously and incrementally rather than as one final recovery pass.

This is **streaming replication**, introduced in PostgreSQL 9.0. Mechanically: the primary runs a background process called the **WAL sender** for each connected standby, and the standby runs a **WAL receiver** process that connects to the primary over a special replication connection and streams WAL records as they are generated, applying them to its own copy of the data pages in near real time.

```
Primary
|
| WAL records, streamed continuously
v
Replica (WAL receiver -> apply)
```

1.4 Primary and replica roles

The primary is the PostgreSQL instance that accepts writes. Every INSERT, UPDATE, and DELETE from the application is directed here, and the primary is the sole generator of the WAL stream for the cluster.

The replica (standby) receives that WAL stream and continuously applies it, arriving at a copy of the primary's data. A standby running with `hot_standby = on` (the default for a standby since PostgreSQL 9.0) can additionally serve read-only queries while it replays incoming WAL, which is what makes a replica useful for offloading read traffic, not just for disaster recovery.

```
Application
|
| writes
v
Primary
|
| WAL stream
v
Replica <---- reads (SELECT)
```

The asymmetry is deliberate and load-bearing: a replica is not a second place writes can go. Directing a write at a standard PostgreSQL streaming replica fails outright, because a standby in recovery mode rejects write transactions by design. (This is distinct from PostgreSQL's separate, explicit logical-replication-based multi-master extensions and third-party solutions, which are a different topic from the default physical streaming replication this module is about.)

1.5 Replication lag: why a replica is never guaranteed to be caught up

Because the WAL stream takes real, nonzero time to travel across the network and be replayed, a replica's applied state is always at or behind the primary's current state, never ahead of it and only instantaneously equal to it under ideal conditions. This gap is **replication lag**, and it can be measured precisely: the primary's current WAL position (an LSN) minus the replica's most recently replayed LSN.

```
Primary write_lsn: 0/1634F850
Replica replay_lsn: 0/1634A200
                    |
                    v
                replica is behind
```

The direct, practical consequence: if the application writes `Order 101 = PAID` on the primary and, in the very next instant, reads `Order 101` from a replica, the replica may not yet reflect that write. This is not a bug or a misconfiguration; it is the correct, expected behavior of asynchronous replication, and it is the reason every architecture that reads from a replica must explicitly decide how much staleness it can tolerate, a decision this document returns to in Module 2.

1.6 Synchronous versus asynchronous replication: the durability-latency trade-off

Section 1.5 assumed the primary commits a transaction and returns success to the client without waiting for any replica's acknowledgment. That is PostgreSQL's default behavior, and it is a deliberate point on a spectrum, not the only option.

ARCHITECT'S DECISION

choosing a `synchronous_commit` level

Options. PostgreSQL's `synchronous_commit` setting, combined with `synchronous_standby_names` naming which standbys are "synchronous," controls exactly what a commit waits for before returning success to the client:

Level	The primary waits for	Data-loss window on primary crash
off	Nothing; WAL is flushed asynchronously in the background	Can lose recently committed transactions that were never even flushed locally
local	The local WAL flush to durable storage on the primary itself	Zero loss on the primary alone, but a promoted standby may still be missing the transaction if it hadn't replicated yet
remote_write	The named synchronous standby to receive and write the WAL to its own operating system (not necessarily fsynced)	A standby-side OS crash before its own fsync could still lose the transaction on that standby
on (the conventional meaning of "synchronous replication")	The named synchronous standby to flush the WAL to its own durable storage	None, once acknowledged; the transaction is durable on at least the primary and the synchronous standby
remote_apply	The named synchronous standby to flush and replay the WAL, making it visible to queries on that standby	None, and additionally guarantees a read against that standby immediately after would see the write

Choice: for Tadka's order and payment data, `synchronous_commit = on` against at least one standby is the right default once a standby exists at all, with `remote_apply` reserved for the specific case where an application explicitly needs read-after-write consistency against a named replica.

Why: the default asynchronous behavior means a promoted standby, after an unplanned primary failure, can be missing the last few transactions the old primary told the client had succeeded, an invisible, silent data-loss window whose size depends entirely on how far behind the standby happened to be at the moment of failure. Synchronous replication converts that variable, unbounded window into a guarantee: nothing the client was told succeeded can be lost by a single-node failure, because it was proven durable on a second node before the client heard "success."

Trade-off: every synchronous level adds a full network round trip (and, for `on` and `remote_apply`, a disk flush or a WAL replay) to the critical path of every commit. This directly increases write latency, and if the named synchronous standby becomes unavailable or slow, by default **every write on the primary blocks** waiting for an acknowledgment that will never come, unless quorum or timeout behavior is explicitly configured. A naive synchronous setup can turn "the standby is having a bad day" into "the entire primary stopped accepting writes."

Failure mode: the specific incident class this creates is the synchronous standby becoming a single point of failure for write availability. PostgreSQL's quorum-based syntax (`synchronous_standby_names = 'ANY 1 (replica_a, replica_b, replica_c)'`, available since PostgreSQL 10) mitigates this by requiring only one of several named standbys to acknowledge, rather than one specific, named standby that might itself be down.

Revisit when: the business articulates an actual zero-data-loss requirement for a specific table or transaction type (payments and order-status transitions are the obvious Tadka candidates), or when the write-latency cost of synchronous replication has been measured against the real order-placement SLA and found acceptable, not assumed acceptable.

1.7 Replication slots: how the primary knows what a replica still needs

Section 1.2 established that WAL segments must be retained until every consumer that needs them has consumed them, and then recycled. Without any coordination, the primary has no reliable way to know how far behind a specific standby is, and historically this was solved with a blunt instrument: retain a generous, fixed amount of WAL (`wal_keep_size`) regardless of whether any standby actually needed it, hoping it was enough.

A **replication slot** solves this precisely instead of by guesswork: it is a named, persistent record on the primary that tracks the oldest WAL position a specific consumer (a specific standby, or a logical-replication consumer) still needs, and the primary guarantees it will not remove WAL segments older than that position, for as long as the slot exists.

```
SELECT * FROM pg_create_physical_replication_slot('replica1_slot');
```

The standby is then configured (via `primary_slot_name` in `postgresql.conf`) to identify itself against that slot when it connects, and the primary now retains exactly what that specific standby needs, no more and no less, as long as the standby is connected and consuming.

The failure mode this specific mechanism introduces, and it is a well-known, production-relevant one: a replication slot's guarantee, "never delete WAL this consumer still needs," holds even if that

consumer disconnects and never comes back. A standby that is decommissioned, crashes permanently, or is simply disconnected for an extended period, while its replication slot remains on the primary, causes WAL to accumulate on the primary's disk without bound, because the primary is faithfully keeping a promise to a consumer that is no longer there to redeem it. This has caused real production outages (the primary's disk filling up entirely, halting all writes) precisely because a slot was left behind after a standby was removed without also dropping its slot. Monitoring `pg_replication_slots` for slots with growing lag, and explicitly dropping slots for decommissioned standbys, is a non-optional operational practice once slots are in use.

1.8 Physical replication versus logical replication

Everything described so far, streaming the raw WAL and replaying it, is **physical replication**: the standby is a byte-for-byte copy of the primary's data pages, cluster-wide, with no ability to replicate only some tables or to run a different PostgreSQL major version on the standby.

Logical replication, introduced in PostgreSQL 10, works differently: it decodes the WAL into a stream of logical row-level changes (this row was inserted, this row was updated, this row was deleted) and applies those changes at the SQL level on the subscriber, using a **publication** (defined on the source, naming which tables to expose) and a **subscription** (defined on the target, naming which publication to consume).

```
-- On the source
CREATE PUBLICATION payment_events FOR TABLE payments;

-- On the target
CREATE SUBSCRIPTION payment_events_sub
    CONNECTION 'host=source-db dbname=tadka'
    PUBLICATION payment_events;
```

ARCHITECT'S DECISION

physical versus logical replication

Options: physical (streaming) replication, or logical replication.

Choice: physical replication for Tadka's read-replica and high-availability topology; logical replication reserved for specific, narrower needs, such as replicating only the menu tables to an analytics database, or performing a near-zero-downtime major-version upgrade.

Why: physical replication gives a complete, exact, cluster-wide copy with the least operational complexity, which is exactly what a read replica or a hot standby for failover needs. Logical replication's selective, row-level model is solving a different problem, moving specific data somewhere else for a specific purpose, not maintaining a general-purpose standby.

Trade-off: logical replication does not automatically replicate schema changes (DDL) or, in most configurations, sequences, and it requires each replicated table to have a primary key or replica identity for updates and deletes to be correctly identified on the subscriber. It is a more surgical, more manual tool. Physical replication, in exchange for its simplicity, cannot filter: it is all of the cluster or none of it, and both sides must run compatible major versions.

Failure mode: attempting to use logical replication as a drop-in substitute for a physical standby, expecting it to transparently handle schema migrations or to serve as a general failover target without

additional tooling, is a common, avoidable design mistake that surfaces the first time a schema change is deployed and silently stops replicating the affected table.

Revisit when: a genuine requirement for selective, cross-version, or filtered replication appears (feeding a subset of tables to a reporting system, migrating between major PostgreSQL versions with minimal downtime), separate from the general-purpose HA and read-scaling requirement physical replication already serves.

1.9 Hot standby and the cost of read queries on a replica

Running read queries against a standby (`hot_standby = on`) is not free of interaction with the replication stream itself. The standby is simultaneously trying to do two things: replay incoming WAL to stay current, and serve read queries against a consistent snapshot of the data as it exists at some point in that replay.

These two goals can conflict. Suppose the primary's vacuum process removes an old row version that a long-running query on the standby still needs to see (for MVCC visibility, covered in depth in Module 8). The standby has two options: cancel the conflicting query, or delay applying that piece of WAL. PostgreSQL's `max_standby_streaming_delay` setting controls how long it will delay replay before giving up and canceling the query, and `hot_standby_feedback`, when enabled, has the standby report its oldest still-needed transaction back to the primary, so the primary's vacuum avoids removing rows the standby still needs in the first place.

The trade-off is symmetric and worth stating precisely: `hot_standby_feedback = on` protects long-running standby queries from being canceled, at the cost of potentially delaying cleanup (vacuum) on the primary, which can increase table and index bloat on the primary if standby queries run for a long time. This is the same category of trade-off as the replication-slot disk-growth risk in Section 1.7: a mechanism that protects one side of a replication relationship by asking the other side to hold onto something for longer.

1.10 Scaling to many replicas: direct fan-out versus cascading replication

A single primary streaming WAL to one or two replicas is unremarkable. The architectural question sharpens once the number of replicas grows.

```

      R1
      R2
      R3
      ...
Primary -----> R50
```

PostgreSQL can technically support this direct topology, every replica connecting straight to the primary, but the primary now pays the cost of running a WAL sender process and a network stream for every single one of those fifty connections, and that fan-out itself becomes a resource the primary must provision for, on top of serving the application's actual write load.

ARCHITECT'S DECISION

direct replication versus cascading replication

Options

Option	Shape	Failure characteristic
Direct	Every replica connects straight to the primary	Simple: any one replica's failure is fully isolated from the others
Cascading	One or a few replicas act as an intermediate relay, themselves streaming WAL onward to further downstream replicas	Reduced primary fan-out, but downstream replicas now depend on the health of their intermediate upstream

Choice: direct replication for Tadka today (a single primary and a single replica); cascading replication understood and held in reserve as a scaling pattern for a future state with a genuinely large number of replicas, not implemented preemptively.

Why: cascading replication solves a specific, measurable problem, primary-side fan-out cost at high replica counts, that Tadka does not currently have. Building a three-level replication topology for a single-replica workload adds operational complexity (more moving parts, more possible failure combinations) in exchange for solving a bottleneck that does not yet exist.

Trade-off: cascading reduces direct load on the primary, but introduces a new dependency chain. If an intermediate replica goes down, every replica downstream of it loses its WAL feed simultaneously, even though the primary itself, and every other branch of the topology, remains completely healthy.

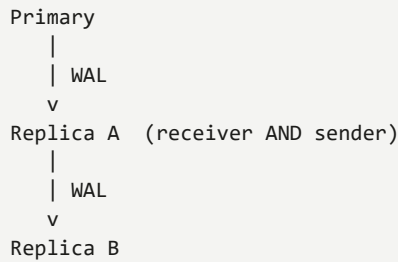
```
Primary ---> R1 (intermediate) ---X DOWN
          |
          +--> R3 (also stops receiving WAL)
          +--> R4 (also stops receiving WAL)
```

Failure mode: treating cascading replication as a free win, "fewer connections to the primary," while overlooking that it has converted an independent-failure topology into a dependent one, is exactly the kind of trade-off this series keeps warning against: every architectural change that solves one problem is worth examining for what new problem it created.

Revisit when: the number of replicas Tadka needs grows to the point where the primary's WAL-sender fan-out is a measured, evidenced bottleneck (CPU or network saturation attributable specifically to replication connections), not a hypothetical future concern.

1.11 Cascading replication in PostgreSQL: how it actually works

The good news, and it is a genuine architectural convenience: cascading replication has been a native PostgreSQL capability since version 9.2, requiring no additional software. A standby can simultaneously be a WAL *receiver* (from its own upstream) and a WAL *sender* (to its own downstream standbys). The downstream standby's `primary_conninfo` (in `postgresql.conf`, since PostgreSQL 12 folded what used to live in `recovery.conf` into the main configuration, activated by the presence of a `standby.signal` file in the data directory) simply points at the intermediate replica's connection details instead of the original primary's.



Production use of this capability still requires the same operational rigor as any replication topology: replication authentication (a dedicated replication role, and `pg_hba.conf` entries permitting replication connections), WAL retention sized correctly at every hop (a replication slot at the intermediate node for each of its own downstream consumers), sender capacity planning, monitoring at every level of the cascade (not just at the primary), and a clear, tested plan for what happens to downstream replicas if an intermediate node needs to be promoted or replaced. The mechanism being native does not make the operational discipline optional.

1.12 Failover and promotion: replication is not automatic failover

This is worth stating with the same force this series used for the equivalent Redis lesson, because it is the identical architectural mistake wearing different technology: **a replica is a copy. Nothing about having one, by itself, decides what happens when the primary dies, or how the application finds the new primary afterward.**

Promoting a standby to become a new primary is a specific, explicit action, historically via `pg_ctl promote` at the operating-system level, or via the `pg_promote()` SQL function available since PostgreSQL 12. Promotion ends the standby's recovery mode and makes it start accepting writes, but it does **not**, by itself:

- tell any other standbys in a cascading topology to re-point themselves at the new primary,
- tell the application, or any connection pooler, which endpoint is now the one to write to,
- guarantee the promoted standby had received every transaction the old primary had already told a client succeeded (this is precisely the asynchronous-replication data-loss window from Section 1.6, now made concrete: whatever WAL had not yet streamed to this standby at the moment of the old primary's failure is simply gone from this standby's history).

This is the exact reason dedicated HA orchestration tools exist (Patroni is the most widely adopted in modern PostgreSQL deployments; `repmgr` and `pg_auto_failover` are other established options). Their job is precisely the job Redis Sentinel does for Redis: continuously monitor the primary's health, reach a reliable, quorum-based decision about whether it has actually failed (as opposed to being merely slow or briefly network-partitioned, which matters enormously, see Section 1.13), execute the promotion, and update whatever mechanism (a virtual IP, a DNS record, a connection-pooler configuration such as HAProxy or pgbouncer, or a proxy layer) the application actually uses to find the current primary.

1.13 Split-brain: the risk a promotion tool exists specifically to prevent

Promotion introduces a distinctive, dangerous failure mode of its own. Suppose the primary is not actually dead, but is merely unreachable due to a transient network partition. An automated failover system, unable to distinguish "the primary is dead" from "I currently cannot reach the primary," promotes

a standby to be the new primary. The network partition then heals, and the original primary, which never crashed and never stopped accepting local connections, is still running and still believes it is the primary.

```
Old Primary (never actually died)
|
+---> still accepting writes

New Primary (promoted during the partition)
|
+---> also accepting writes
```

Two nodes, both believing themselves to be the sole writer, both accepting writes independently: this is **split-brain**, and it is a direct, severe correctness incident, capable of producing two divergent histories of the same data that cannot be automatically, losslessly reconciled.

The standard mitigation is **fencing** (sometimes phrased, colorfully but accurately, as STONITH, "shoot the other node in the head"): the failover mechanism must not merely promote a new primary, it must also guarantee the old primary can no longer accept writes, whether by forcibly stopping it, revoking its network access, or some equivalent hard guarantee, before the new primary is trusted. A promotion mechanism that does not fence the old primary has not actually solved the failover problem; it has merely added a new way for the failover problem to become worse.

1.14 Disaster recovery: point-in-time recovery and WAL archiving

Replication protects against a single-node failure while the cluster is running. It does not, by itself, protect against a mistake (an accidental `DROP TABLE`, a bad migration) that a healthy standby will faithfully and immediately replicate right alongside the primary. That is a distinct concern: **disaster recovery**, and PostgreSQL's mechanism for it is **point-in-time recovery (PITR)**.

The two ingredients are a **base backup** (a full, physical copy of the data directory, typically taken with `pg_basebackup`) and **continuous WAL archiving** (`archive_mode = on`, with an `archive_command` that copies each completed WAL segment somewhere durable, commonly object storage). Recovery then means restoring the base backup and replaying archived WAL forward to any desired target, an LSN, a timestamp, or a named recovery point, stopping exactly at the moment before the mistake occurred.

```
Base backup (Monday, 2 a.m.)
|
v
Replay archived WAL: Monday 2 a.m. --> Monday 2:47 p.m.
|
v
Stop just before the accidental DROP TABLE at 2:48 p.m.
```

In production, this is rarely hand-rolled from raw `pg_basebackup` and `archive_command` scripting; dedicated tools (`pgBackRest`, `WAL-G`, and `Barman` are the established choices) manage backup scheduling, compression, retention, and the replay orchestration, because getting PITR right by hand, and more importantly, **verifying it actually works by periodically rehearsing a real restore**, is exactly the same discipline this series established for any backup: an untested restore path is a hypothesis, not a disaster recovery plan.

1.15 Managed PostgreSQL: what the platform gives you, and what it still doesn't

A managed offering such as Azure Database for PostgreSQL Flexible Server provides infrastructure-level replication capability out of the box, including cross-region read replicas and zone-redundant high-availability configurations (typically implemented as a synchronous standby within the same region, promoted automatically on primary failure), and typically automates PITR through managed backups.

The distinction that must not be lost: a managed service manages *infrastructure*. It does not make *architecture decisions* on your behalf. You still, unavoidably, have to decide and be able to defend:

```
How many replicas, and where (region, zone)?
|
v
Direct topology, or cascading?
|
v
What is the acceptable replication lag for each read path?
|
v
Synchronous or asynchronous commit, for which tables?
|
v
How does the application discover and route to the current primary after a failover?
|
v
What is the tested, verified recovery time and recovery point objective?
```

Managed PostgreSQL reduces **operational burden**: you are not the one patching the WAL sender or provisioning the failover orchestration software. It does not reduce **distributed-systems complexity**: every question above still has to be answered by whoever is designing the system on top of the managed service, in exactly the same way a managed Redis offering (Module 1 of the companion Redis reference) reduces Sentinel-operation burden without eliminating the need to understand what Sentinel is deciding on your behalf.

1.16 Applying this to Tadka

Tadka's current topology is deliberately minimal:

```
Primary :5432
|
v
Replica :5433
```

One primary, one replica, direct (not cascading) replication, asynchronous by default. Given the four-question test this document formalizes in Module 3, is a cascading topology justified here? No: there is exactly one replica, so there is no fan-out problem for cascading to solve. Building Primary -> Replica1 -> Replica2 -> Replica3 today, with no current need for three replicas, would only add dependency chains (Section 1.10's trade-off) in exchange for solving nothing. Cascading replication in this document, as with the Redis Cluster discussion in the companion reference, is a pattern to *recognize and be ready to reach for*, not a default to apply preemptively.

1.17 Extended scenario bank

Scenario 1. A standby has been disconnected for three days after a network configuration error, and its replication slot is still present on the primary. The primary's disk usage has grown by 40GB over that period, with no corresponding increase in application write volume. What is the most likely explanation, and what is the fix? *Answer:* the replication slot is faithfully retaining every WAL segment generated since the standby disconnected, because the slot's guarantee holds regardless of whether the standby is actually consuming it (Section 1.7). The fix is either restoring connectivity so the standby catches up and the slot's retained-WAL watermark advances, or, if the standby is genuinely being decommissioned, explicitly dropping the slot so the primary is free to recycle that WAL.

Scenario 2. After an unplanned primary failure and automatic promotion of the standby, three orders that customers were shown a successful confirmation for are missing from the new primary. The system was running with default replication settings. What happened, and what setting would have prevented it? *Answer:* this is the asynchronous-replication data-loss window (Section 1.6): those three transactions had been committed and acknowledged to the client on the old primary but had not yet streamed to the standby at the moment of failure, so the promoted standby has no record of them. `synchronous_commit = on` with an appropriate `synchronous_standby_names` configuration would have made the primary wait for the standby to durably receive each transaction before acknowledging it to the client, closing this specific window at the cost of added write latency.

Scenario 3. Two engineers are debating a failover incident. One argues "the standby was promoted, so we're fully recovered." The other insists something is still wrong. A few minutes later, writes to the old primary's IP address (which nobody had redirected) start appearing, and the application's data becomes inconsistent. What went wrong, and what category of tooling exists specifically to prevent it? *Answer:* this is split-brain (Section 1.13): the old primary was never fenced off from accepting writes, so once anything (a stale connection pool, a cached DNS entry, a client that hadn't yet been redirected) resumed sending it traffic, both nodes were independently accepting writes. Dedicated HA orchestration tooling (Patroni and similar) exists specifically to fence the old primary as an atomic part of the promotion, not as an optional afterward step.

Scenario 4. A team wants to replicate only the `menu` and `restaurants` tables to a separate analytics database, without giving the analytics team access to payment or order data, and while running a newer PostgreSQL major version on the analytics side than production uses. Which replication mechanism fits, and why would the default streaming-replication approach not work here? *Answer:* logical replication (Section 1.8) is the correct fit, because it supports table-level selectivity (a publication can name specific tables) and can replicate across different major versions, neither of which physical streaming replication supports, since physical replication always copies the entire cluster byte-for-byte between identical major versions.

Scenario 5. A standby configured with `hot_standby_feedback = off` is repeatedly canceling a long-running analytics query that used to complete successfully. Around the same time, the primary's table bloat has visibly decreased after a recent tuning change. Are these two observations related, and what is the trade-off involved in fixing the query cancellations? *Answer:* yes, they are directly related (Section 1.9): with `hot_standby_feedback` off, the primary's vacuum process is free to remove old row versions the standby's long-running query still needs, causing cancellation; a recent change that made vacuum more aggressive (reducing bloat) has made this worse. Enabling `hot_standby_feedback` would stop the cancellations, at the cost of the standby holding back the primary's vacuum for as long as that query runs, reintroducing some of the bloat the recent tuning change was trying to reduce, a direct trade-off between standby query reliability and primary-side table bloat.

Module 2

Read Replicas in Application Architecture

2.1 Why add a read replica at all

A read replica exists to move read load off the primary, freeing the primary's capacity for writes and for reads that genuinely require the primary's guarantees. This is only a real benefit if reads are actually the bottleneck, which is a diagnosis question, not an assumption, addressed directly in Section 2.4.

2.2 Read-after-write consistency: the problem replicas create

The moment any read is allowed to go to a replica, Section 1.5's replication lag becomes an application-visible correctness question, not just a database-internals detail. A customer places an order, the write commits on the primary, and the customer's very next action, viewing the order, is a read. If that read is routed to a replica that has not yet replayed the WAL for that write, the customer sees either a 404 Not Found or a stale prior state, a directly confusing, support-ticket-generating experience, even though nothing in the system is actually broken.

2.3 Strategies for consistency-sensitive reads

ARCHITECT'S DECISION

handling read-after-write consistency

Options

Strategy	Mechanism	Cost
Route the read to the primary	Any read that immediately follows a write by the same request or session is sent to the primary instead of a replica	The primary absorbs exactly the reads that most need consistency, but this reintroduces load onto the primary for precisely the reads an architect was hoping to offload
Session/request affinity	Once a session has written, pin that session's subsequent reads to the primary (or to a specific, tracked replica) for a bounded window	Simple to reason about per-user, but requires the routing layer to track session-to-node affinity, adding statefulness to what is otherwise a stateless routing decision
Track replica LSN and wait or route accordingly	The write path returns the LSN of its own commit; a subsequent read either waits until a replica's <code>replay_lsn</code> has caught up to that LSN, or is routed only to a replica already known to have reached it	Precise and correct, but requires plumbing the LSN through the application and the routing layer, real engineering effort for a real guarantee
Accept eventual consistency	Reads may briefly reflect a slightly stale state; no special handling	Zero implementation cost, but only acceptable where a customer seeing "not yet updated" for a second or two is genuinely tolerable

Choice: for the majority of Tadka's read traffic (menu browsing, historical order lists after the immediate placement moment), eventual consistency against a replica is entirely acceptable. For the single specific read that follows immediately after `POST /orders` within the same request or the same user session, route to the primary.

Why: applying the strongest, most expensive consistency strategy uniformly across every read defeats the purpose of having a replica at all; applying no strategy anywhere produces exactly the confusing, intermittent "my order isn't there" experience Section 2.2 described. The correct answer is per-read-path, not global.

Trade-off: routing "the read right after a write" to the primary means the primary must be sized to handle that specific traffic shape, which is smaller than "all reads" but not zero, and the routing layer (an API gateway rule, a connection-pooler policy, or application-level logic) now carries a small amount of read/write-awareness it would not otherwise need.

Failure mode: the most common mistake is applying this rule too broadly, for instance by routing an entire user session's reads to the primary for its whole lifetime "just to be safe" after a single write, which quietly reintroduces most of the read load the replica was added to absorb, defeating the replica's purpose while looking, on a diagram, like the problem was solved correctly.

Revisit when: the specific read paths that need strong consistency change (a new feature displays order status immediately after an update from a different service, for instance), or when LSN-tracking infrastructure already exists for another reason and the precision it offers becomes cheap to reuse here as well.

2.4 Scale the bottleneck, not the symptom

Before adding a replica at all, the correct first step is diagnosis, not provisioning. A primary at 85% CPU utilization under heavy read traffic is not automatically evidence that a read replica is the fix; it might just as easily be a missing index, an inefficient ORM-generated query, an undersized connection pool causing contention, or a large, unbounded scan somewhere in the read path. Reaching for a replica before running `EXPLAIN ANALYZE` on the actual slow queries risks solving the wrong problem expensively: the replica now also runs the same badly-written query, just with more hardware behind it, and the underlying inefficiency is still there, still costing resources, merely spread across more machines.

```
High CPU on primary
  |
  v
Diagnose: bad query? missing index? connection pool? locking?
  |
  v
Fix the actual bottleneck first
  |
  v
THEN consider a read replica, cache, or further scaling
```

The general principle: **a read replica is a scaling tool for a genuinely read-bound, well-optimized workload, not a substitute for query and index tuning.**

2.5 Extended scenario bank

Scenario 1. After adding a read replica specifically to offload the `GET /orders` endpoint, the primary's CPU utilization barely changes. What are the two most likely explanations to investigate first? *Answer:* either the application's routing layer is not actually sending `GET /orders` traffic to the replica (a configuration or connection-string issue, worth confirming with `pg_stat_activity` on both nodes), or the primary's high CPU was never dominated by that specific query in the first place, meaning the diagnosis in Section 2.4 was skipped and the replica was added to solve an assumed bottleneck rather than a measured one.

Scenario 2. A support ticket reports that a customer's order confirmation page briefly showed "order not found" immediately after checkout, then worked correctly on refresh a few seconds later. What is the most likely architectural cause, and which of Section 2.3's strategies directly addresses it? *Answer:* the confirmation-page read was routed to a replica that had not yet replayed the WAL for the order-creation write (Section 2.2's read-after-write consistency gap). Routing the immediate post-write read to the primary (Section 2.3's first strategy) directly closes this specific gap without requiring every read in the system to become primary-routed.

Scenario 3. A team proposes routing every read for a logged-in user to the primary "to be completely safe," for the lifetime of the user's session. What would you push back with? *Answer:* this defeats the purpose of having a replica at all for the vast majority of reads that do not follow a recent write, reintroducing most of the load the replica exists to absorb (Section 2.3's failure-mode warning). The correct scope is the specific read that follows a specific write, not an entire session by default.

Module 3

The Service Extraction Decision Framework

3.1 The question that should never be answered with "because microservices"

Tadka's Payment capability, by this point in its evolution, already lives inside a modular monolith: a single deployed process with a clean internal boundary around Payment's logic, state, and gateway integration. The question this module answers is not whether that boundary is well designed; it already is. The question is whether that *logical* boundary should become a *physical, network* boundary, a separate deployable service.

The wrong way to answer this question is to start from the technology: "let's extract Payment because separate services are the modern approach," or "because Payment is an important domain." Those statements describe an aesthetic preference, not a problem. **The right way to answer it starts from naming a specific, current problem the existing boundary cannot solve, and then checking whether extraction is genuinely the cheapest way to solve it.**

3.2 Testing the weak reasons

"Payment is slow"

Payment's gateway latency was already a real, measured problem, and it was already solved without touching the process boundary at all: a request timeout bounded how long any single call could wait, a bulkhead bounded how much concurrency the gateway integration could consume, and moving payment processing off the synchronous request path removed the gateway's latency from the customer-facing response entirely. Every one of those fixes happened *inside* the existing monolith. Since the latency problem is demonstrably solvable without a network boundary, "Payment is slow" cannot, by itself, be the justification for creating one; whatever the real justification is, it has to be a different problem.

"Microservices are best practice"

This is the most common and most dangerous justification precisely because it sounds like an established convention rather than an argument. A domain boundary (Payment as a distinct concept with its own logic and its own data) is genuinely valuable, and it is entirely achievable inside a modular monolith, with a clean internal package structure and enforced internal references, without paying for a network hop, a second deployment pipeline, or a second database to operate. A service boundary is one specific, expensive way to express a domain boundary; it is not the only way, and it is not automatically the better way.

"We need to scale Payment independently"

This becomes a legitimate reason only once it is backed by evidence, not merely by the fact that independent scaling is *possible* once a boundary exists. If Order traffic runs at 100 requests per second and Payment traffic runs at 10, there is no scaling asymmetry to exploit yet. If Payment traffic instead runs at ten times Order's rate, the case strengthens considerably. The architect's question is never "can a separate service scale independently," since the answer to that is trivially yes; it is **"does the evidence show Payment's actual scaling profile is different enough from the rest of the system to**

justify operating it separately?"

3.3 Testing the legitimate reasons

Independent deployment

If Payment integrations change frequently, and the team that owns those changes has a release cadence genuinely different from the rest of the system, coupling every payment-logic change to the same deployment pipeline as unrelated order or restaurant features is a real, ongoing cost. This becomes a strong reason once the organizational fact (different team, different cadence, different risk tolerance for deploys) is actually true, not merely theoretically possible if the org were structured differently.

Security and compliance boundary

Payment logic sits near card data, tokens, gateway credentials, and financial records, categories of data that typically carry stricter access, auditing, and retention requirements than the rest of the system. Extracting Payment into its own service, with its own database, its own access controls, and its own deployment pipeline, can meaningfully shrink the blast radius of who and what can touch that sensitive data. This must be stated precisely, though: **a separate service is a tool that can help reduce compliance scope and tighten access control; it is not, by itself, a compliance certification.** Actual regulatory compliance (PCI DSS, for a payments context) depends on what data is stored, how it is transmitted, who has access, what logging and monitoring exist, and a range of infrastructure and process controls that a service boundary alone does not satisfy.

Fault isolation

This is the strongest and most concrete of the legitimate reasons, and it is worth stating exactly why. Inside a single process, a catastrophic failure in Payment's code, an unhandled exception that crashes the process, a memory leak that exhausts available resources, has the structural potential to take the entire process down with it, including capabilities (menu browsing, viewing existing orders) that have nothing to do with payment.

```
Tadka Process
|
+-- Order
+-- Payment <-- catastrophic failure here
+-- Restaurant
|
v
Entire process can go down
```

A separate Payment process changes this structurally, not just statistically: a crash in the Payment service's process cannot, by the operating system's own process-isolation guarantees, directly crash the Order service's process.

```
Order Service      Payment Service
|                  |
|                  CRASHES
|
+--> remains running
```

3.4 The four-question test

Whenever the question "should we extract this into its own service" is raised, whether by a teammate, a manager, or your own instinct, apply this test before answering:

Question 1: What problem are we actually solving? Name it precisely. "Fault isolation for Payment failures" and "a stronger security and data-ownership boundary around payment-sensitive information" are precise, testable claims. "Payment feels important" and "this is how modern systems are built" are not.

Question 2: Does that problem actually exist today, or is it a hypothetical future concern? This is the most consistently underrated question in architecture discussions. A future possibility and a present, evidenced fact are not the same category of justification, and only the latter should drive an expensive, hard-to-reverse structural change today.

Question 3: Can the monolith solve it, and at what cost if it can? For fault isolation specifically, the honest answer is that a modular monolith can isolate *code and logic* cleanly, but it structurally cannot isolate *process-level* failure the way a separate operating-system process can; a crash is a crash for everything sharing that process. For data ownership, the monolith *can* enforce a strict rule that only Payment's own code touches Payment's own tables, but that rule is a convention enforced by discipline and code review, not a hard boundary enforced by the operating system and the network, the same "architecture test versus folder convention" distinction this series applies to module boundaries generally.

Question 4: What new failure modes does extraction introduce, and are they smaller than the problem being solved? A function call inside one process becomes a network call between two. That single change brings an entire category of new failure with it: timeouts, connection refusals, DNS resolution failures, partial responses, retries that might duplicate work, and a debugging experience that now spans two independently deployed, independently logged processes instead of one call stack. None of this is free, and all of it must be weighed honestly against whatever the extraction was meant to buy.

3.5 Why yesterday's async decision makes today's extraction safer

This is a specific, important, and easy-to-miss connection between two decisions made at different points in the system's evolution. Before payment processing was moved off the synchronous request path, the customer-facing `POST /orders` call would have depended, directly and synchronously, on Payment succeeding. Had Payment been extracted into a separate network service at that earlier point, any unavailability of the Payment service would have propagated immediately and directly into a failed order-creation request, because Order was still waiting on Payment's response before it could answer the customer at all.

Because payment processing had *already* been moved to an asynchronous, background path before extraction was considered, the extraction inherits that same decoupling for free: Order can still return 201 Created with a `PaymentPending` status even while the newly separate Payment service is completely unreachable, because the request path was never depending on Payment's synchronous response to begin with.

```
If extraction had come first:
Order --(synchronous HTTP)--> Payment --X DOWN
|
v
Order request fails too

Given the actual order of decisions:
Order --> Created --> 201
|
v
Background Payment attempt --X DOWN
|
v
Order API is unaffected
```

The general architectural lesson: a decision made for one reason (removing gateway latency from the request path) can quietly become the precondition that makes a *later*, seemingly unrelated decision (extracting that capability into its own service) safe rather than dangerous. Architecture decisions are rarely independent of each other, and recognizing which earlier decisions a proposed change is quietly depending on is itself an architect-level skill.

3.6 Extended scenario bank

Scenario 1. A junior engineer argues: "Payment already has its own classes, its own interface, and its own database tables inside the monolith, so extracting it into a service is basically free, we're just moving files." What is missing from this reasoning? *Answer:* it conflates a logical (code-level) boundary, which is real and valuable, with a physical (network and process) boundary, which introduces an entirely new category of failure (Section 3.4's fourth question) that has nothing to do with how the code is currently organized. Moving well-organized code across a network boundary is a smaller, more controlled change than moving badly-organized code, but it is never "basically free."

Scenario 2. A team has evidence that Payment's traffic pattern is genuinely different from Order's (Payment spikes sharply during promotional events in a way Order does not), but the same single team owns both capabilities and deploys them together on the same weekly schedule. Which of the legitimate reasons from Section 3.3 applies, and which does not yet? *Answer:* independent scaling has real supporting evidence and is a legitimate technical reason to consider extraction. Independent deployment does not yet apply, since the same team and the same release cadence mean there is no organizational friction that separate deployability would actually relieve; that specific reason would need to wait for genuine evidence of a deployment-cadence mismatch, not just a scaling mismatch.

Scenario 3. After extracting Payment purely for fault isolation, an incident review finds that a Payment service crash *did* still bring down Order API responses, because Order was calling Payment synchronously and waiting on its response before answering the customer. What does this reveal about the extraction, independent of whether the extraction itself was justified? *Answer:* it reveals that Section 3.5's precondition, payment processing already being decoupled from the synchronous request path, was not actually in place before extraction happened. Extraction alone does not guarantee fault isolation for the caller if the caller still makes a blocking, synchronous call to the now-separate service; the isolation benefit specifically depends on the caller's own resilience design, not merely on the callee being a separate process.

Module 4

Designing the Boundary

4.1 What actually moves: ownership, not just code

Extracting Payment is not the exercise of copying some classes into a new repository and standing up a new deployment. It is the exercise of assigning clear, exclusive **ownership**: which capability, which state, and which data belongs entirely to the new service, with nothing else in the system permitted to touch it directly.

```
Payment Service owns:
  Payment business logic
  Payment transaction state
  Payment gateway integration
  Payment credentials and configuration
  Payment persistence (its own database)

Order Service owns:
  Order
  Order lifecycle and status
  Order creation
  The client-side logic for requesting payment
```

The single most important consequence of this ownership split: **Order Service does not query Payment's database, under any circumstance.** Every interaction between the two happens through an explicit, versioned API that Payment Service controls and can evolve independently, exactly the way Module 4 of the companion resilience reference frames the same principle for module boundaries generally: a boundary that can be bypassed by a direct database query, or a direct class reference, is not actually a boundary.

4.2 The interface-first principle, and its cross-stack equivalents

Business logic inside Order Service should never be written directly against an HTTP client. It should be written against an abstraction that expresses the *capability* being used, with the transport mechanism hidden entirely behind it.

.NET:

```

public interface IPaymentClient
{
    Task<PaymentResult> ChargeAsync(
        ChargePaymentRequest request,
        CancellationToken cancellationToken);
}

public sealed class PaymentClient : IPaymentClient
{
    private readonly HttpClient _httpClient;

    public PaymentClient(HttpClient httpClient) => _httpClient = httpClient;

    public async Task<PaymentResult> ChargeAsync(
        ChargePaymentRequest request,
        CancellationToken cancellationToken)
    {
        // HTTP call to the Payment Service lives here, and only here
        var response = await _httpClient.PostAsJsonAsync(
            "/payments", request, cancellationToken);
        return await response.Content
            .ReadFromJsonAsync<PaymentResult>(cancellationToken);
    }
}

```

Java: a `PaymentClient` interface backed by a Feign client or a `RestClient`-based implementation, with domain code depending only on the interface.

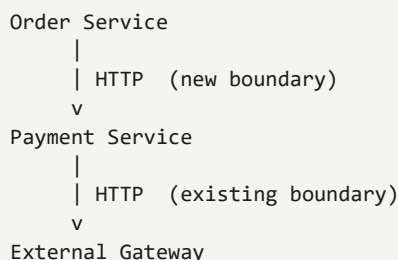
Node.js: a `PaymentClient` module exposing a small, typed function surface, with an axios or fetch-based implementation hidden behind it.

Go: a `PaymentClient` interface satisfied by an HTTP-backed implementation struct, injected into whatever code needs to charge a payment.

The principle underneath every one of these is identical, and it is worth stating as its own rule, independent of language: **business code should depend on a contract expressing what capability it needs, never on the transport mechanism that happens to fulfill it today**. If tomorrow the Payment Service moves from REST to gRPC, or is temporarily replaced by a local stub during a migration, the code calling `IPaymentClient.ChargeAsync` does not need to change at all.

4.3 Two resilience boundaries and the timeout-budget problem

Before extraction, Tadka's payment path had exactly one network dependency to protect against: the call from the payment worker out to the external gateway. After extraction, there are two, layered on top of each other:



Each of these boundaries needs its own timeout, its own bulkhead, and, if warranted, its own circuit breaker, because each protects against a *different* failure: Order Service needs protection against Payment Service being slow or unavailable, and separately, Payment Service needs its own, independent protection against the external gateway being slow or unavailable. These are not the same failure, and treating them as one by applying a single timeout value at only one layer misses the failure that occurs at the other layer entirely.

ARCHITECT'S DECISION

allocating the timeout budget across a call chain

Options: apply the same timeout value at every hop uniformly, or reason about the end-to-end budget explicitly and allocate it across hops.

Choice: treat the end-to-end latency budget (derived from Order Service's own SLA to its caller) as the total to be allocated, and give each hop a portion of it, not a copy of it.

Why: the caller's timeout has to account for everything that happens underneath it: the network round trip to Payment Service, whatever Payment Service itself spends processing and calling the gateway, and the return trip. If Order Service's timeout to Payment Service is set to the same value Payment Service uses for its own call to the gateway, Order Service's timeout can never actually be reached in the specific case where Payment Service's own gateway call is the thing running long, because Order's clock and Payment's clock for the "same" operation are not measuring the same span of time; Order's span fully contains Payment's span, plus its own overhead.

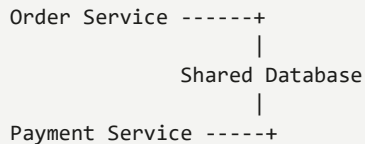
Trade-off: reasoning about a shared budget across independently owned, independently deployed services requires those teams to actually communicate about their respective timeout values, which is an ongoing coordination cost that a single-process monolith never had to pay, because a single timeout in one place used to be sufficient.

Failure mode: copying a timeout value from one layer to the next without doing this arithmetic is a common, subtle bug: Order Service's 2-second timeout to Payment Service, combined with Payment Service's own 2-second timeout to the gateway, means the true worst case the customer can experience is closer to 4 seconds (Payment's full timeout elapsing, plus network overhead, before Order's own timeout even has a chance to fire on a clean, fast failure), not the 2 seconds the diagram misleadingly suggests at a glance.

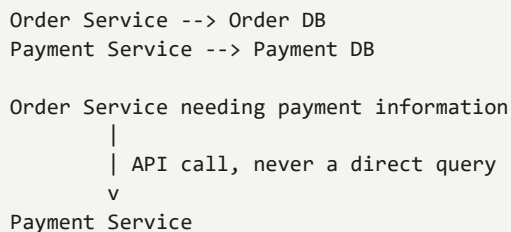
Revisit when: a new hop is added anywhere in the chain, or when either service's own SLA changes, since a per-hop timeout that was correct for one topology can silently become wrong the moment the topology grows by even one more hop.

4.4 Database-per-service, and why a shared database breaks the boundary

Suppose, for convenience during extraction, Payment Service and Order Service are pointed at the same underlying database instance.



This looks, superficially, like two services. It is not a real boundary, because nothing prevents Order Service from executing `UPDATE payments SET status = 'PAID' WHERE order_id = 101;` directly, bypassing every validation rule, every audit-trail entry, and every business invariant that Payment Service's own code exists to enforce. **A service boundary drawn without a corresponding data boundary is decorative.** The correct topology gives each service its own database, with the only sanctioned path between them being the API:



This is not free. Section 3.4's fourth question applies directly: a database-per-service topology surrenders the simple, local, single-transaction guarantee that a shared database provided for free. Creating an order and recording that a payment succeeded used to be one atomic database transaction; across two databases, it is not, and the system now needs an explicit strategy (events, an outbox, a saga, reconciliation) for a consistency guarantee the old topology gave away without anyone having to design for it. This exact trade-off, and the mechanisms that address it, is the subject of Module 6.

4.5 Extended scenario bank

Scenario 1. A code review flags that `OrderService` has a method that builds an HTTP request and calls `httpClient.PostAsync("http://payment-service/payments", ...)` directly inside a controller action. What principle from this module does this violate, and what is the concrete fix? *Answer:* it violates the interface-first principle from Section 4.2: business logic (here, the controller handling order creation) is directly coupled to the transport mechanism (raw HTTP) instead of to a capability abstraction. The fix is introducing an `IPaymentClient` interface, moving the HTTP-specific code into an implementation of it, and having the controller depend only on the interface.

Scenario 2. Order Service's timeout to Payment Service is set to 2 seconds. Payment Service's own timeout to the external gateway is also set to 2 seconds. During an incident, customers experience order-creation requests taking nearly 4 seconds before failing. Using Section 4.3, explain why. *Answer:* the two timeouts are not coordinated against a shared budget; they are each independently sized as if they were the only hop in the chain. In the worst case, Payment Service's full 2-second timeout to the gateway elapses before it can respond to Order Service at all, and only then does Order Service's own 2-second clock (which started when it first called Payment Service) have a chance to matter, compounding rather than bounding the total wait.

Scenario 3. A team extracts Payment but keeps a foreign key from the `orders` table directly to the `payments` table within the same physical database, arguing "it's more efficient than an API call for reporting queries." What boundary problem does this create, independent of query efficiency? *Answer:* it

recreates Section 4.4's shared-database failure mode in a subtler form: any code with database access to that shared instance, including future reporting jobs, ad hoc scripts, or a different team entirely, can bypass Payment Service's own logic and read or write payment data directly. The ownership boundary this module insists on is only real if the database itself is not shared, regardless of how convenient a cross-table join might be for one specific reporting use case.

Module 5

Proving Fault Isolation

5.1 A claim is not a proof

Having designed and implemented the extraction, it is tempting to declare victory: the architecture diagram shows two separate boxes, therefore fault isolation exists. This is exactly the gap this series repeatedly warns against: **a diagram is an assertion, not a proof**. The only way to know whether a boundary actually behaves the way it was designed to is to deliberately break the dependency and observe what actually happens.

5.2 Establishing the baseline

Before breaking anything, confirm the happy path genuinely works end to end: both services report healthy on their respective health-check endpoints, and a real `POST /orders` request flows all the way through order creation, the asynchronous payment attempt, the HTTP call to Payment Service, the gateway interaction, and back to a final successful payment state. Skipping this step means any later failure observed during the fault-injection test is ambiguous: is it revealing something about fault isolation, or is it merely an unrelated bug in the happy path that was never actually working?

5.3 Killing the dependency: what should, and should not, fail

With Payment Service deliberately stopped, three specific tests establish whether the new boundary behaves as designed, in increasing order of how interesting the answer is.

Test 1: `GET /menu`. This capability has no dependency on Payment whatsoever. The expected, and only acceptable, result is `200 OK`, completely unaffected. If this fails while Payment is down, the "boundary" was never a real boundary, it was a shared fate dressed up as separate services.

Test 2: `GET /orders (existing orders)`. Reading previously created orders should not require a live call to Payment Service; that data already lives entirely within Order Service's own database. The expected result is again `200 OK`, served entirely from Order DB, with Payment Service's outage having no visibility into this code path at all.

Test 3: `POST /orders (creating a new order)`. This is the test that actually exercises the fault-isolation design from Section 3.5. Because payment processing happens asynchronously, in the background, after the customer-facing response has already been returned, Order Service can, and should, still return `201 Created` with a status of `PaymentPending`, even while Payment Service is completely unreachable. The user-facing capability, placing an order, survives the dependency's outage; only the payment's eventual completion is affected, and only for as long as the outage lasts.

Payment Service: DOWN

```
GET /menu          -> 200 OK (unaffected)
GET /orders        -> 200 OK (unaffected)
POST /orders       -> 201 Created, status = PaymentPending
                   (payment itself is delayed, not the order)
```

If all three tests pass, the extraction has genuinely delivered the fault isolation it was justified by in Module 3. If any of them fail, the boundary looks correct on a diagram but is not correct in practice, and the specific failing test identifies exactly where the coupling actually lives.

5.4 The architecture review checklist: five common mistakes

Beyond the specific fault-injection tests above, a small set of recurring design mistakes are worth actively reviewing for whenever a new service boundary is drawn, because each one produces a diagram that *looks* like a clean microservices architecture while quietly failing to deliver the isolation the diagram implies.

Mistake 1: the shared database. Covered in depth in Section 4.4. The tell is any arrow from a service that does not "own" a table directly into that table.

Mistake 2: introducing a message broker before there is a reason for one. A diagram that routes every interaction through Kafka, "for scalability," when a simple, synchronous HTTP call was already sufficient for the actual current volume and consumer count, has added an entire new category of operational surface (brokers, topics, partitions, consumer groups, offset management, ordering and duplicate-delivery questions) without a corresponding requirement driving it. The correct sequencing is always: identify a specific requirement (durable delivery, high throughput, multiple independent consumers, replay) first, and let that requirement justify the broker, rather than adopting the broker and then discovering, or failing to discover, whether the requirement was ever really there.

Mistake 3: a synchronous chain across multiple services. A design where `Order -> Payment -> Notification -> Delivery`, each one calling the next synchronously and waiting on its response, means a failure anywhere in that chain propagates backward through every link, and the service at the front of the chain (Order, in this case) becomes a hostage to the reliability of every service behind it, including ones with no direct business relationship to the customer-facing request it is trying to fulfill.

Mistake 4: "one service per table." Deciding service boundaries by counting database tables, an `Orders` table becomes an Order Service, a `Restaurants` table becomes a Restaurant Service, mistakes an implementation detail (how data happens to be normalized in the current schema) for an architectural principle. Service boundaries should track business capability, ownership, failure isolation, and scaling or security characteristics; a table count is a coincidental correlation with some of those factors, not a substitute for reasoning about them directly.

Mistake 5: assuming a distributed transaction can replace a local one. Before extraction, "create the order and record the payment" could be one local database transaction, atomic by construction. After Payment and Order have separate databases, that same guarantee cannot be recreated by simply wrapping both calls in something labeled a "transaction"; it requires an explicit strategy, exactly the subject of Module 6, and pretending otherwise produces a design that looks consistent on paper and silently loses that consistency the first time a failure occurs mid-operation.

5.5 Extended scenario bank

Scenario 1. During a fault-isolation test, `GET /menu` unexpectedly returns a 500 error while Payment Service is stopped. Investigation shows the menu endpoint's health-check middleware pings every downstream service, including Payment, before serving any request. What category of mistake from Section 5.4 does this resemble, even though it is not literally a shared database? *Answer:* it is a variant of the synchronous-chain mistake (Mistake 3): an unrelated capability has been made to synchronously

depend on Payment's availability, this time through health-check middleware rather than a direct business call. The fix is scoping health checks (and any other cross-cutting middleware) to only the dependencies a given request path actually needs, rather than treating "all services must be healthy" as a blanket precondition for serving any request.

Scenario 2. A post-incident review finds that during a real Payment Service outage, `POST /orders` did correctly return `201 Created`, but a separate internal admin dashboard that displays "orders awaiting payment confirmation" also went down at the same time, because it synchronously calls Payment Service to look up each order's current status. Does this invalidate the fault-isolation claim from Module 3? *Answer:* not for the customer-facing order-placement capability the extraction was specifically justified to protect, since Test 3 in Section 5.3 still passed. It does reveal a separate, previously unreviewed synchronous dependency (Mistake 3) in the admin dashboard that was never included in the original fault-isolation analysis, a useful, narrower finding: fault isolation has to be verified for every consumer of the affected dependency, not assumed to extend automatically to capabilities that were not explicitly tested.

Scenario 3. A new engineer proposes: "since Order and Restaurant each have their own table already, let's also split Restaurant into its own service to match Payment's pattern." What question from this module should be asked before agreeing? *Answer:* Section 5.4's Mistake 4 directly applies: having a separate table is not, by itself, a reason to create a separate service. The question to ask is the same four-question test from Section 3.4, applied fresh to Restaurant: what specific problem would extracting Restaurant solve, does that problem exist today, can the monolith solve it, and what new failure modes would extraction introduce, rather than extracting it merely because it superficially resembles the Payment precedent.

Module 6

The Durability Wound: Space Versus Time

6.1 What extraction did, and did not, solve

Module 5 proved that extraction delivered genuine fault isolation: a Payment Service outage no longer takes the Order API down with it. This is a real, structural improvement, separation **in space**, two independent processes that can fail independently of one another. It is worth being precise about exactly what this improvement does not cover.

Suppose Payment Service is down for thirty seconds. Almost certainly fine. Suppose it is down for thirty minutes. Orders continue to be created, each one landing in a `PaymentPending` state, with a background worker attempting, and failing, to reach the now-unavailable Payment Service for each one. When Payment Service eventually comes back, what happens to those thirty minutes' worth of pending orders?

6.2 Why RAM is not enough

The current implementation's background payment-processing mechanism holds its pending work as an in-memory job, inside the process's own RAM. This has a dangerous property, once stated plainly: **anything that destroys that process's memory destroys the pending work along with it**, and a failed HTTP call to an unavailable Payment Service is exactly the kind of event that can cause the in-memory job to be dropped rather than durably retained for a later attempt.

```
Order 101 created
  |
  v
In-memory payment job
  |
  v
Worker attempts HTTP call
  |
  X
Connection refused (Payment Service is down)
  |
  v
Job may simply be gone
```

Restart the worker process for any reason, a routine deploy, an autoscaling event, a crash, and every job that existed only in that process's memory at that moment is gone, regardless of whether Payment Service itself is healthy again by the time the worker restarts. The failure does not require anything dramatic; an ordinary, routine process restart is sufficient to lose the work.

6.3 Three options for making the work durable

ARCHITECT'S DECISION

where does the pending payment work actually live?

Options

Option	Mechanism	Durability	Cost
In-memory retry	Keep the job in RAM; retry a bounded number of times on failure before giving up	None beyond a single process's lifetime; a crash or restart during the retry window loses the job entirely	Lowest; no new infrastructure
Database-backed job table	Persist the pending payment as a row (e.g., in a <code>PaymentJobs</code> table), written in the same transaction as the order itself	Survives a worker crash or restart, since the job row exists independently of any single process's memory	Moderate; the existing database now does double duty as a queue, and at very high volume this becomes its own bottleneck (polling load, lock contention, table growth)
Dedicated message broker	Publish the payment work as a durable message (Kafka, RabbitMQ, Azure Service Bus) consumed by one or more independent workers	Strong; the broker is purpose-built for durable delivery, and typically supports multiple independent consumers and replay	Highest; a new piece of infrastructure to run, monitor, and reason about, along with its own delivery, ordering, and duplicate-message concerns

Choice: for Tadka's current scale, a database-backed job table is the right next step beyond the in-memory approach, precisely because the database already exists, already provides transactional guarantees, and already provides the specific property this module needs, an order and its associated pending-payment record can be created as one atomic transaction, closing the gap between "the order exists" and "the fact that it needs payment exists" that the pure in-memory approach left open.

Why: a dedicated broker is the correct answer to a different, more advanced set of requirements, specifically high throughput, multiple independent consumers, message replay, or genuinely decoupled asynchronous workflows, none of which Tadka's current payment volume actually demands yet. Reaching for Kafka because "durable async processing" sounds like a Kafka-shaped problem, without first checking whether the requirement is actually there, is exactly the "technology before the problem" mistake Section 5.4 already warned against for message brokers generally.

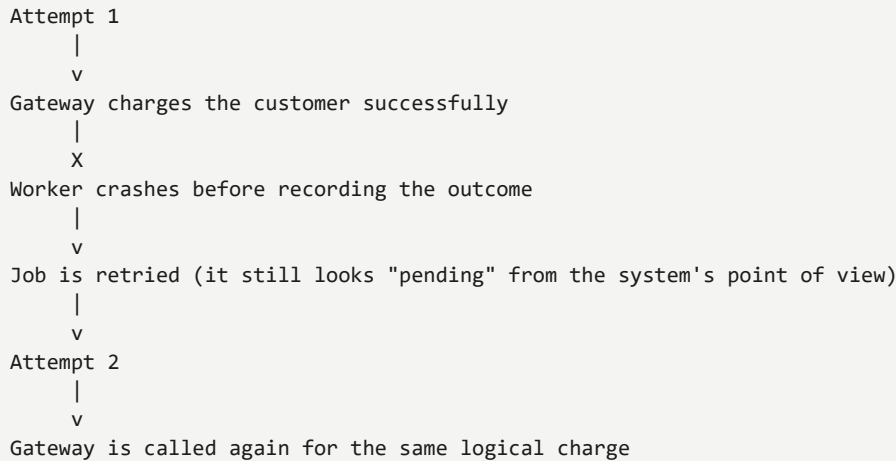
Trade-off: a database-backed job table means the same database that serves the application's core transactional workload also absorbs polling reads and writes for the job queue. At high enough volume, this can produce lock contention on frequently-pollled rows, an ever-growing table that needs its own cleanup strategy, and coordination complexity once multiple worker instances are polling the same table concurrently.

Failure mode: treating "we added a database table" as equivalent to "we solved durability completely" stops one layer too early. The job surviving a crash is necessary but not sufficient; Section 6.4 covers the next problem this creates.

Revisit when: the actual, measured payment volume and worker count start to make the job table itself a bottleneck (visible as query contention or growing table-scan costs specifically tied to job-polling queries), or when a genuine requirement for multiple independent consumers of the same payment-lifecycle events appears, at which point the case for a dedicated broker becomes evidence-based rather than aspirational.

6.4 Idempotency returns, and why it must

Making the job durable solves "can the work survive a crash." It does not solve a closely related, equally dangerous problem: what happens if the *gateway* successfully processes a charge, but the *worker* crashes before it manages to record that success?



Without protection, this can charge the customer twice for one order. The fix, exactly as established for the equivalent problem in the payment-resilience reference, is an **idempotency key**: a stable identifier for this specific logical operation (for example, `PAY-ORDER-101`), sent with every attempt, that the gateway is expected to recognize on a retry and treat as "already handled," returning the original result rather than executing the charge a second time, within whatever semantics that specific gateway provider guarantees for repeated idempotency keys.

The broader lesson this reinforces: **durability and idempotency are two separate, complementary requirements, not one problem with one fix**. Durability answers "can the work survive a crash and be attempted again." Idempotency answers "is it safe to attempt the same work more than once." A system needs both; either one alone leaves a real gap.

6.5 The architecture progression

It is worth holding the entire evolution in view as one sequence, because each version solved one problem while surfacing the next, and no version was a wrong or wasted step, each was the correct response to the evidence available at that point.

```
Version 1: Order -> Payment -> Gateway (fully synchronous)
    Problem: gateway latency directly slows down order placement

Version 2: Order -> Payment Worker (in-memory, asynchronous) -> Gateway
    Problem: the pending work exists only in process memory

Version 3: Order Service -> (network) -> Payment Service -> Gateway
    Problem: fault isolation improved, but the asynchronous work is
    still not durable across the new network boundary

Version 4 (the next step, not built in this module):
    Order -> Durable job (DB-backed, or eventually an outbox/broker)
        -> Payment Worker -> Payment Service -> Gateway
        + Idempotency
        + Retries
        + Reconciliation for outcomes that remain unknown
```

6.6 Extended scenario bank

Scenario 1. A team fixes the durability wound by adding a `PaymentJobs` table, but continues to insert a row into it from application code running *after* the order has already been committed, in a separate transaction. During a crash-injection test, some orders end up with no corresponding payment job at all. What was the actual mistake? *Answer:* creating the order and creating its payment job in two separate transactions reopens a narrower version of the exact gap durability was meant to close: a crash between the two commits leaves an order with no record that it ever needed a payment attempt. The fix is inserting both rows within the same database transaction, exactly as Section 6.3 specifies, so the two facts are either both durably recorded or neither is.

Scenario 2. After introducing a database-backed job table and idempotency keys, a customer is still charged twice, but this time the gateway's own logs show it correctly rejected the duplicate idempotency key on the second attempt. Where should the double charge actually be traced to? *Answer:* since the gateway correctly deduplicated at its layer, the double charge is happening somewhere upstream of the gateway call, most likely two different idempotency keys were generated for what should have been treated as the same logical payment attempt (for instance, a bug that derives the key from a retry-specific value instead of a stable, order-specific one), which defeats the entire purpose of the idempotency key even though every individual component appears to be working as designed.

Scenario 3. A stakeholder asks why the team did not simply add Kafka as part of fixing the durability wound, given that "Kafka is the standard tool for this kind of problem." How would you respond, using this module's reasoning? *Answer:* point to the actual, current requirement: at Tadka's present volume, a single database-backed job table already provides the specific property needed, durable, transactionally-consistent pending work, without the operational cost of running and monitoring a broker. Kafka becomes the right answer once a concrete, evidenced requirement (very high throughput, multiple independent consumer types, a need for replay) actually exists, and adopting it ahead of that evidence repeats the "technology before the problem" mistake this document has warned against throughout.

Module 7

Query and Schema Design at Scale

7.1 Indexing: workload-specific, not universal

An index is not a generic performance switch to flip on; it is a targeted optimization for a specific, known query pattern, purchased at a specific, ongoing cost.

ARCHITECT'S DECISION

adding an index to a large, slowing table

Options: add a single-column index on the most obvious filter column; add a composite index matching the actual query shape; do not index and address the bottleneck elsewhere.

Choice: for a query filtering on `customer_id` and `status` together and sorting by `created_at`, a composite index built in that order, `(customer_id, status, created_at DESC)`, is the correct fit, not a lone index on `customer_id` alone.

Why: PostgreSQL can use a composite index to satisfy an equality filter on the leading columns and simultaneously return rows in the index's own physical order for the trailing sort column, avoiding a separate, expensive sort step after the rows are found. A single-column index on `customer_id` alone would still require PostgreSQL to filter by `status` and sort by `created_at` after fetching every one of that customer's rows, which is a meaningfully more expensive plan once a customer has thousands of historical orders. This should always be confirmed against the actual query plan with `EXPLAIN ANALYZE`, rather than assumed from the schema alone.

```
CREATE INDEX idx_orders_customer_status_created
ON orders (customer_id, status, created_at DESC);
```

Trade-off: every index makes every write to that table marginally more expensive, since the index itself must be updated on every insert, update, or delete that touches an indexed column, and every index consumes storage and competes for the database's cache memory alongside the actual table data.

Failure mode: the common overcorrection, once a team internalizes "indexes are good," is creating a separate index for every distinct query pattern the application happens to issue. With fifty different query shapes, fifty targeted indexes is very likely the wrong answer: the cumulative write-amplification and memory pressure from that many indexes can become a larger cost than the read-latency problem any single one of them solves. The architect's discipline is identifying the *dominant*, highest-value access patterns and indexing deliberately for those, not reflexively indexing every column that ever appears in a `WHERE` clause.

Revisit when: `EXPLAIN ANALYZE` on the actual production query shows a sequential scan or an expensive sort where an index scan should be possible, or when write latency on a heavily-indexed table has measurably degraded and an audit of index usage (via `pg_stat_user_indexes`) reveals indexes that are rarely, if ever, actually used by the query planner.

7.2 Pagination: offset versus keyset

ARCHITECT'S DECISION

paginating a 100-million-row table

Options: offset-based pagination (`OFFSET n LIMIT 20`), or keyset (cursor-based) pagination.

Choice: keyset pagination for any deep, sequential paging experience ("next page" style browsing); offset pagination reserved for cases where users genuinely need to jump to an arbitrary page number.

Why: `OFFSET 1000000 LIMIT 20` requires PostgreSQL to still identify and skip past the first one million matching rows before it can return the next twenty, and that skip cost grows linearly with the offset, meaning page 50,000 is dramatically more expensive to retrieve than page 5. Keyset pagination instead carries forward the last-seen row's sort key (typically paired with a unique tiebreaker to remain stable when multiple rows share the same primary sort value) and asks the database for rows strictly after that point, which the index can satisfy directly regardless of how deep into the dataset the client has paged:

```
SELECT *
FROM orders
WHERE customer_id = 123
  AND (created_at, id) < (:lastCreatedAt, :lastId)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Trade-off: keyset pagination cannot directly answer "show me page 500," since it only knows how to move forward or backward from a known position, not how to compute an arbitrary offset. If the product requirement genuinely includes jumping to an arbitrary page number, that specific UX requirement has to be weighed explicitly against keyset's performance advantage, rather than one approach being unconditionally correct.

Failure mode: applying keyset pagination's cursor logic without a stable, unique tiebreaker (sorting only by `created_at`, which is not guaranteed unique) can silently skip or duplicate rows whenever two rows share the exact same timestamp, a subtle correctness bug rather than a performance one.

Revisit when: a genuine product requirement for direct page-number navigation is stated explicitly, at which point the trade-off between UX and query cost has to be made deliberately, for example by capping how deep offset-based jumps are allowed to go.

7.3 Identifier design: UUID, UUIDv7, and sequences

ARCHITECT'S DECISION

choosing a primary key type for a large, high-write table

Options: a random (version 4) UUID; a time-ordered (version 7) UUID; a database-generated sequential integer (`BIGINT/IDENTITY`).

Choice: for Tadka's `orders` table, a sequential `BIGINT` identifier for internal use, with a separate, non-sequential, externally-exposed order reference if hiding internal volume or ordering from customers or partners matters.

Why: global uniqueness and index locality are two different properties, and a random UUID satisfies only the first. Because a version-4 UUID's bytes are effectively random, each new row's key lands in an

essentially random position within the primary key's B-tree index, rather than at the end of it, which causes far more scattered page writes and page splits than a monotonically increasing key does, degrading both insert throughput and the index's on-disk locality as the table grows large. A sequential integer, or a UUIDv7 (which encodes a timestamp in its leading bits specifically to preserve rough insertion-order locality while remaining globally unique), avoids this by keeping new rows clustered near the end of the index, where B-tree insertions are cheapest.

Trade-off: a purely sequential integer key, if ever exposed externally (in a URL, an API response), reveals the approximate row count and creation order of the table to anyone who can see it, an information leak that matters more in some domains (order volume, potentially commercially sensitive) than others. UUIDs, including UUIDv7, avoid this specific leak at the cost of a larger key size (16 bytes versus 8) that modestly increases index storage and memory footprint compared to a `BIGINT`.

Failure mode: choosing random UUIDs purely for the "globally unique, no coordination needed" property, without accounting for the index-locality cost, is a well-documented, easy-to-fall-into performance mistake that only becomes visible once the table has grown large enough for B-tree fragmentation to actually matter, well after the identifier scheme is already deeply embedded throughout the schema and difficult to change.

Revisit when: cross-shard or cross-database uniqueness becomes a real requirement (a scenario Module in this document's sharding discussion returns to), or when externally exposing sequential identifiers is judged an unacceptable information disclosure for the specific business domain.

7.4 Extended scenario bank

Scenario 1. A composite index on `(customer_id, status, created_at DESC)` exists, but `EXPLAIN ANALYZE` on the target query still shows a sequential scan. What are the first two things to check before assuming the index is simply not being used? *Answer:* first, confirm the query's actual filter and sort clauses match the index's column order and direction exactly, since a mismatched sort direction or an additional filter on a column not covered by the index can prevent the planner from using it as expected; second, check whether the table's statistics are stale (via `ANALYZE`) or whether the query planner has simply, correctly, decided a sequential scan is cheaper for this specific case, which can genuinely happen when a filter matches a very large fraction of the table, making an index scan more expensive than just reading everything sequentially.

Scenario 2. Product asks for a "jump to page 200" feature on the customer order history screen, which currently uses keyset pagination. What would you propose, given Section 7.2's trade-off? *Answer:* propose a hybrid: keep keyset pagination as the default, fast mechanism for normal sequential browsing, and offer offset-based jumps only up to a bounded, reasonable depth (for instance, capping direct page-number access to the first few thousand rows), explicitly communicating that deep, arbitrary page jumps are either unsupported or noticeably slower, rather than silently allowing an unbounded offset query that could become a performance incident under real usage.

Scenario 3. A team switches from random UUIDs to UUIDv7 for a high-write table and observes a measurable improvement in insert throughput, without changing anything else. What does Section 7.3 predict caused this, specifically? *Answer:* UUIDv7's leading timestamp bits keep newly inserted rows clustered together near the end of the primary key's B-tree, closely mirroring the locality benefit of a sequential integer key, whereas the previous random UUIDs scattered new rows across the entire index, causing more page splits and less cache-friendly access patterns; the throughput improvement is the direct, predictable consequence of restoring that locality.

Module 8

Concurrency Control

8.1 MVCC and the lost-update problem

PostgreSQL uses Multi-Version Concurrency Control (MVCC): rather than locking a row for every reader, each transaction sees a consistent snapshot of the data as it existed at a defined point, and concurrent writers create new row versions rather than blocking readers. This gives excellent read concurrency, and it does not, by itself, protect against the classic **lost update**: two transactions each read the same value, each independently decide on a new value based on what they read, and the second one to commit silently overwrites the first one's change with no indication that a conflict ever occurred.

```
Both read quantity = 10

Transaction A writes 9   (based on reading 10)
Transaction B writes 8   (based on reading 10, unaware of A's write)

Final value: 8
Transaction A's update is silently lost
```

Whether this is acceptable depends entirely on the business requirement, never on the database mechanism alone; "last write wins" is PostgreSQL's default behavior for two independent, non-conflicting-at-the-row-lock-level updates, and it is often the wrong behavior for the business.

8.2 Optimistic versus pessimistic locking

ARCHITECT'S DECISION

preventing lost updates on a frequently-contended row

Options

Optimistic locking: carry a version number (or a timestamp) with every read, and make the write conditional on that version still matching:

```
UPDATE orders
SET quantity = 9, version = 11
WHERE id = 101 AND version = 10;
```

If this affects zero rows, the application knows another transaction updated the row in between, and can decide to retry, re-read, or surface a conflict to the user.

Pessimistic locking: acquire an explicit row lock at the moment of reading, blocking any other transaction from reading (in the locking sense) or writing that row until the current transaction finishes:

```
SELECT * FROM orders WHERE id = 101 FOR UPDATE;
```

Choice: optimistic locking is the right default whenever conflicts are relatively rare, which describes the overwhelming majority of Tadka's row-level update patterns. Pessimistic locking is reserved for the narrower case of genuinely high contention on a specific, hot row, where operations must be strictly serialized and holding a lock for the duration is an acceptable cost.

Why: optimistic locking never blocks a reader and only pays a retry cost in the comparatively rare case of an actual conflict, whereas pessimistic locking pays a blocking cost on every single access to the locked row, whether or not a conflict would actually have occurred that time.

Trade-off: optimistic locking requires the application to correctly handle the "zero rows affected" case, retrying or surfacing a conflict, additional logic that pessimistic locking's blocking behavior avoids needing. Pessimistic locking, in exchange for that simplicity, risks holding locks for longer than intended if the transaction that acquired the lock is slow, itself a path toward the deadlock risk covered next.

Failure mode: choosing pessimistic locking by default, "to be safe," on a table with a high read-to-write ratio and low actual contention needlessly serializes access that never needed to be serialized, turning what should be a highly concurrent workload into an artificially sequential one.

Revisit when: measured conflict rates under optimistic locking become high enough that the retry cost itself becomes the bottleneck, at which point pessimistic locking, or a redesign that reduces contention on the hot row directly (for instance, splitting a single contended counter into several partial counters), becomes worth the added blocking cost.

8.3 Deadlocks and lock ordering

Suppose Transaction A locks Order 1 and then attempts to lock Order 2, while Transaction B, running concurrently, has already locked Order 2 and is now attempting to lock Order 1. Neither transaction can proceed: each is waiting on a lock the other one holds. PostgreSQL detects this cycle and resolves it by aborting one of the two transactions, returning a deadlock error to whichever client issued it.

The database's detection and resolution is a safety net, not a design. The architect's actual question is upstream of the database entirely: why did the application allow two transactions to acquire the same two resources in a different order in the first place? The standard, effective mitigation is enforcing a single, deterministic lock-acquisition order across the entire application, for example, always locking rows in ascending order of their primary key, regardless of which business operation is doing the locking, so that two transactions touching the same two rows can never form a cycle. Complementary practices, keeping transactions as short as possible, never making an external network call (to a payment gateway, for instance) while holding a database lock, and using the least restrictive isolation level the correctness requirement actually needs, all reduce the *probability* of contention turning into an actual deadlock, even with correct lock ordering in place.

8.4 Extended scenario bank

Scenario 1. A checkout flow uses optimistic locking on the `orders` table, and under a specific promotional flash-sale event, the retry rate on this update spikes dramatically, degrading overall checkout latency even though no individual conflict is expensive to resolve. What does Section 8.2 suggest reconsidering? *Answer:* this is precisely the "revisit when" trigger from Section 8.2: a sudden, sustained spike in contention on one specific row (or a narrow set of rows) is the signal that optimistic locking's retry cost has itself become the bottleneck. The fix is not necessarily switching to pessimistic locking outright; it may instead mean redesigning the hot row itself, for instance decomposing a single

contended inventory counter into several partitioned sub-counters that are summed on read, reducing how often independent transactions actually collide on the same physical row.

Scenario 2. Two different code paths both update an order and a related payment record, but one path locks the order row first and then the payment row, while the other path locks the payment row first and then the order row. What specific, latent risk does this create, and what is the general-purpose fix?

Answer: this is a textbook deadlock setup (Section 8.3): under the right timing, one transaction from each path can end up each holding the lock the other one needs. The general fix is enforcing one deterministic, application-wide lock acquisition order (for example, "always lock the lower-numbered ID first" using a stable, comparable identifier across both tables) so both code paths are structurally incapable of forming a locking cycle, regardless of which one happens to run first.

Scenario 3. A developer proposes wrapping a payment-gateway HTTP call inside the same database transaction that also updates the order's status, reasoning "this way it's all atomic." What concern from this module should be raised immediately? *Answer:* holding a database transaction, and therefore any row locks it has acquired, open for the duration of an external network call is a serious anti-pattern: the transaction's duration is now at the mercy of a third-party gateway's latency (exactly the kind of dependency the resilience-pattern material treats as unpredictable and occasionally very slow), during which every lock the transaction holds blocks every other transaction that needs those same rows, dramatically increasing both lock-wait time and deadlock probability across the whole system for as long as the external call is in flight.

Module 9

Caching, Sharding, and the Payment Resilience Recap

9.1 Cache-aside and cache stampede

The default, and usually correct, caching pattern is cache-aside: check the cache first, and on a miss, read from PostgreSQL and populate the cache for next time.



A **cache stampede** occurs when a single, hot cache entry expires and a large number of concurrent requests miss it simultaneously, all falling through to PostgreSQL at once to independently recompute the same value. The mitigations, covered in full depth in the companion Redis reference, include a single-flight lock so only one request rebuilds the value while the rest wait or receive a stale value, TTL jitter so many keys populated at the same moment do not all expire in the same instant, and stale-while-revalidate serving for read paths where a few seconds of staleness is genuinely invisible to the user. The architectural point worth restating here specifically in a PostgreSQL context: **the actual danger of a cache stampede is not the cache miss itself, it is the database absorbing a sudden, synchronized burst of load it was never provisioned for**, precisely the same cascading-failure shape as an outright cache outage.

9.2 Cache invalidation strategy

ARCHITECT'S DECISION

keeping a cached menu consistent with the database

Options: rely on TTL expiry alone; explicitly update the cache on every write; explicitly delete the cache entry on every write, letting the next read repopulate it; drive invalidation from a database change-event stream.

Choice: delete-on-write, combined with a TTL as a safety net.

Why: deleting the cache entry the moment the underlying data changes bounds the staleness window to "until the next read," rather than "until the TTL expires," which is a meaningfully tighter and more predictable guarantee for a menu update a restaurant expects customers to see promptly. The TTL remains as a backstop against any invalidation call that is missed due to a bug or a deployment gap, so staleness never becomes permanent even if the explicit invalidation path fails once.

Trade-off: delete-on-write requires every code path capable of writing to the underlying data to also remember to invalidate the cache, a discipline that a purely TTL-based approach does not require but which comes at the cost of the data always being at most-TTL stale, never fresher.

Failure mode: a common, easy-to-miss bug is a secondary write path, a bulk import script, a direct administrative update, that changes the underlying data without going through the same code path responsible for cache invalidation, leaving the cache silently stale until the TTL eventually expires it, precisely the case the TTL safety net exists to bound.

9.3 Sharding strategies and the hot shard problem

ARCHITECT'S DECISION

sharding a 5-billion-row orders table

Options: range-based sharding (by date, or by ID range), hash-based sharding ($\text{hash}(\text{customer_id}) \% N$), tenant-based sharding (one or more shards dedicated to a specific customer), geographic sharding.

Choice: hash-based sharding on `customer_id`, using a shard-mapping approach (consistent hashing, or an explicit shard-assignment table) rather than a naive modulo.

Why: hashing on `customer_id` distributes a typical customer base's load reasonably evenly, and keeping all of a given customer's rows on the same shard avoids needing cross-shard joins for the overwhelmingly common "show this customer's own orders" query pattern.

Trade-off: a naive $\text{hash}(\text{customer_id}) \% N$ scheme ties the shard assignment directly to the current shard count N ; changing N (adding capacity) changes the modulo result for a large fraction of existing keys, forcing a massive data-movement event just to add one more shard. Consistent hashing, or an explicit, versioned mapping table, decouples "which shard does this key belong to" from "how many shards exist today," so growing the shard count moves only a small, bounded fraction of the data rather than nearly all of it.

Failure mode, the hot shard. Uniform key distribution is not the same guarantee as uniform *workload* distribution. If a single enterprise customer generates thirty percent of Tadka's total order volume, hashing correctly places every one of that customer's rows on exactly one shard, and that shard now carries thirty percent of the system's real traffic no matter how evenly the hash function itself behaves. The fix is recognizing this as a distinct problem from key distribution and addressing it directly: dedicated infrastructure or sub-sharding specifically for known large tenants, rather than assuming a well-distributed hash function has automatically solved workload balance.

Revisit when: a single shard's resource utilization is measurably and persistently disproportionate to its peers, the concrete symptom of a hot shard, independent of whether the overall hash distribution looks statistically even.

9.4 Payment resilience recap

The full technical depth for these patterns, timeout selection, bulkhead sizing, circuit-breaker state machines, and rate limiting, is covered in the companion resilience reference; the essential shape is restated here because it is directly load-bearing for this document's service-extraction narrative.

Timeout. Bounds how long the caller waits before reclaiming its own resources; sized from the caller's own SLA, never blindly copied from a downstream dependency's documented latency, and, once a call chain has multiple hops (Section 4.3), allocated as a shared budget across those hops rather than duplicated at each one.

Unknown outcome. A timeout tells you only that you stopped waiting; it does not tell you whether the operation actually succeeded on the other side. For payments specifically, this ambiguity must be resolved through idempotency keys, an explicit payment state machine (including a legitimate "unknown, pending reconciliation" state), and, eventually, a reconciliation process, never through a blind retry alone.

Bulkhead. Bounds how much concurrent capacity a single dependency is allowed to consume, so a slow payment gateway cannot exhaust the resources every other capability in the system also depends on.

Circuit breaker. Stops calling a dependency that is currently, demonstrably failing, protecting the caller's own resources from being spent on calls very unlikely to succeed; it does not repair the dependency, it only contains the caller's exposure to it.

9.5 Extended scenario bank

Scenario 1. A menu cache uses delete-on-write invalidation, but a data-migration script that bulk-updates thousands of menu items runs directly against the database, bypassing the application's normal write path entirely. What is the predictable consequence, and how long does it last? *Answer:* every menu item touched by the migration becomes stale in the cache, since the script's writes never trigger the delete-on-write invalidation logic that lives in the application layer; the staleness persists until each affected key's TTL naturally expires, which is exactly the scenario Section 9.2 identifies the TTL safety net as bounding, rather than eliminating.

Scenario 2. A shard rebalancing operation, adding a fourth shard to a three-shard hash-based topology using naive modulo, is scheduled to run during a known high-traffic event because "it needs to happen eventually." What would you recommend instead, and why? *Answer:* recommend rescheduling the resharding operation to a known low-traffic window, and, more fundamentally, moving toward a consistent-hashing or explicit shard-mapping scheme so that future capacity additions do not require a similarly disruptive, large-scale data movement at all; performing a naive modulo-based reshard during peak traffic compounds the operation's inherent data-movement cost with the worst possible timing.

Scenario 3. A single enterprise customer's shard is consistently running at far higher CPU utilization than every other shard, even though the hash function's overall key distribution, measured across the full customer base, looks statistically even. What should be concluded, and what should not? *Answer:* this should be recognized as the hot shard problem (Section 9.3), not treated as evidence the hash function is broken; the hash function is very likely distributing keys correctly, but one key happens to carry a disproportionate share of the actual workload. The correct response is targeted, for instance dedicating additional infrastructure to that specific tenant or sub-sharding its own data further, not re-hashing the entire dataset in search of a distribution problem that does not actually exist.

Module 10

The Architect's Meta-Framework

10.1 The seven questions to ask before designing anything

Across every module in this document, from WAL internals to service extraction to query design, the same small set of questions recurs underneath the specific technical details. Training yourself to ask these explicitly, before reaching for any specific technology, is the actual transferable skill this entire reference has been building toward.

1. What is the actual problem? Not "which technology should we use," but the precise, underlying need the technology would be serving.

2. What is the scale? Concretely: requests per second, data volume and growth rate, expected concurrency, typical payload size, and the ratio between peak and average load, since a design correct for the average case can fail entirely at the peak.

3. What are the actual guarantees required? Consistency, availability, latency, durability, ordering, and whether "exactly-once" business effect (not merely exactly-once message delivery, a materially harder and often unnecessary guarantee) is truly needed.

4. What can fail? For every single arrow in an architecture diagram, from one component to another, ask explicitly what happens if the target is slow, what happens if it is completely unavailable, what happens if the response is lost in transit after the target actually processed the request, and what happens if the target processed the request but the caller has no way of knowing that.

5. What does this decision cost? Every architectural choice carries a benefit, a cost, a new failure mode it introduces, and an ongoing operational burden; naming only the benefit is an incomplete analysis, not a decision.

6. How will we know it is actually working? Metrics, logs, traces, an explicit service-level objective, alerts tied to that objective, and capacity measurements taken from real, not assumed, data.

7. How does this architecture evolve? The question developers most often skip, and the one that most reliably separates senior engineering from architecture: what happens when this system is ten times its current size, and what happens when today's core assumption turns out to be wrong.

10.2 The skill-progression framework

Evaluating architectural maturity is not about whether someone can draw a diagram that looks correct. It is about which of these levels their reasoning actually operates at, across each of the dimensions below.

Dimension	Junior Developer	Senior Developer	Architect
Requirements	Implements what is given	Clarifies some ambiguity	Actively challenges ambiguous or untested requirements
Scale	Estimates vaguely	Calculates explicitly	Connects the calculated numbers directly to architectural choices

Dimension	Junior Developer	Senior Developer	Architect
Technology	Knows the available tools	Chooses among the tools	Can explain why a popular tool should specifically not be used here
Database	Designs a schema	Optimizes queries against it	Designs for both consistency guarantees and future schema evolution
Failure	Handles exceptions as they arise	Adds resilience patterns	Deliberately designs failure boundaries and blast radii
Distributed systems	Recognizes known patterns	Applies patterns correctly	Understands and can articulate a pattern's specific consequences
Services	Creates microservices readily	Defines clear boundaries	Justifies, with evidence, whether a service should exist at all
Performance	Optimizes code	Optimizes individual components	Locates the actual, current bottleneck before optimizing anything
Security	Implements specified controls	Designs authentication and authorization	Defines and defends the system's actual trust boundaries
Architecture	Draws diagrams	Designs working systems	Defends the trade-offs behind the design, under challenge
Production	Fixes reported bugs	Owens a service's operation	Owens the system's behavior end to end, including its failure modes
Business alignment	Limited visibility into business context	Understands business impact	Directly connects architectural decisions to business outcomes

10.3 The one sentence worth internalizing

An architect is not the person who knows the most technologies. An architect is the person who can make a defensible decision under real constraints, explain its trade-offs honestly, and understand precisely what happens when that decision turns out to be wrong.

Every module in this document, the WAL's durability guarantee, the choice between direct and cascading replication, the four-question test for service extraction, the durability wound extraction exposed, the indexing and sharding decisions, exists to make that one sentence something you can actually do, not merely something you can recite.

Appendix A: The System Design Question Bank, Developer to Architect

This appendix consolidates twenty-four question-and-answer topics into a single interview-preparation reference. Each entry follows the same structure: the question as it might realistically be posed, the answer a competent but junior developer would likely give, the deeper reasoning an architect would apply instead, and the general principle the topic distills. Every answer here is written to be technically precise on its own, independent of which module it also happens to connect to elsewhere in this document.

A.1 Database indexing

Question. An Order API serves `GET /orders?customerId=123&status=DELIVERED` against a table of one hundred million rows, using the query `SELECT * FROM orders WHERE customer_id = 123 AND status = 'DELIVERED' ORDER BY created_at DESC LIMIT 20;`. The endpoint is getting slower as the table grows. What would you do?

Developer answer. Add an index on `customer_id`.

Architect thinking. Before creating any index, establish the actual query pattern, how selective `customer_id` and `status` are individually and together, whether the `ORDER BY` clause is actually required by the product, what `EXPLAIN ANALYZE` reports for the current query plan, and how frequently this specific query actually runs relative to the table's write volume. The likely correct index is a composite one matching the full access pattern, `CREATE INDEX idx_orders_customer_status_created ON orders(customer_id, status, created_at DESC);`, but the decision being made is not "composite indexes are generally better." It is that this specific index matches this specific dominant access pattern, letting PostgreSQL filter and return the newest twenty matching rows without a separate, expensive sort step.

Architect follow-up. If the application has fifty distinct query patterns against the same table, the correct response is not fifty corresponding indexes. Every index has a read benefit, a storage cost, a write-amplification cost, a memory and cache-pressure cost, and an ongoing maintenance cost, and the architect's job is identifying which access patterns are actually dominant enough to deserve a dedicated index.

Architect principle. An index is a workload-specific optimization, not a generic performance improvement that should be applied reflexively.

A.2 Offset, cursor, and keyset pagination

Question. Tadka has one hundred million orders, and the API needs to support `GET /orders?page=50000&pageSize=20`. Would you implement this with offset-based pagination?

Developer answer. Yes, using `OFFSET 1000000 LIMIT 20`.

Architect thinking. At deep page numbers, the database still has to locate and skip past every one of the preceding rows before it can return the requested twenty, making the cost of retrieving page fifty thousand dramatically higher than page five. For an ordered dataset, keyset pagination is the correct default: `SELECT * FROM orders WHERE customer_id = 123 AND created_at < :lastCreatedAt ORDER BY created_at DESC LIMIT 20;`, ideally with a stable tiebreaker added to the comparison, `WHERE`

`(created_at, id) < (:lastCreatedAt, :lastId) ORDER BY created_at DESC, id DESC LIMIT 20;`, so that rows sharing an identical timestamp are still handled deterministically.

Architect follow-up. If product genuinely requires the ability to jump directly to an arbitrary page, such as page five hundred, the decision changes, and now has to explicitly balance the user-experience requirement against query performance, data consistency during pagination, and implementation complexity, rather than treating one pagination technique as universally correct.

Architect principle. Do not select a pagination technology before understanding the actual product requirement it needs to satisfy.

A.3 UUIDs and index locality

Question. Every order ID is a randomly generated UUID, and the table holds five hundred million rows. Would you use that UUID as the table's primary, clustered index?

Developer answer. Yes, because a UUID is globally unique.

Architect thinking. Uniqueness and index locality are two entirely different properties. Because a standard, random (version 4) UUID's bytes are effectively random, each newly inserted row lands at an unpredictable position within the primary key's B-tree, rather than appending near the end of it, causing more page splits and worse on-disk locality than a value that increases predictably with insertion order. The available options include a plain UUID, a time-ordered UUID (version 7), a plain sequential `BIGINT`, or a database-generated sequence, and the right choice depends on evaluating uniqueness requirements, key distribution, index size, locality, any sharding requirements, whether the identifier will ever be exposed externally, and how much guessability of that identifier matters for security.

Architect principle. Choose identifiers based on the actual workload and system boundaries the identifier has to serve, not on uniqueness as the sole criterion.

A.4 MVCC and concurrent updates

Question. Two users simultaneously modify the same order's quantity. Both read the current value of ten. User A writes nine. User B writes eight. What happens, and is that acceptable?

Developer answer. Last write wins, so the final value is eight.

Architect thinking. Whether that outcome is acceptable is a business question, not a database mechanics question, and it should never be assumed away. If losing an update silently is unacceptable, optimistic locking makes the write conditional on the row's version not having changed since it was read (`UPDATE orders SET quantity = 9, version = 11 WHERE id = X AND version = 10;`, treating zero affected rows as a detected conflict), or pessimistic locking acquires an explicit row lock at read time (`SELECT * FROM orders WHERE id = ? FOR UPDATE;`), serializing access to that row for the duration of the transaction.

Architect decision. Use optimistic locking when conflicts are relatively uncommon, since it never blocks a reader and only pays a cost when an actual conflict occurs. Use pessimistic locking when contention is genuinely high, the operation must be strictly serialized, and holding a lock for the operation's duration is an acceptable cost.

Architect principle. Locking strategy should follow the business's actual consistency requirements and the workload's real contention characteristics, not a default habit applied uniformly.

A.5 Deadlocks

Question. Transaction A locks Order 1 and then attempts to lock Order 2. Transaction B, running concurrently, locks Order 2 and then attempts to lock Order 1. What happens?

Architect answer. A deadlock occurs. PostgreSQL detects the circular wait and aborts one of the two transactions to break the cycle. The deeper architectural question is why the application allowed two transactions to acquire the same two resources in different orders in the first place. The standard fix is enforcing a single, deterministic lock-acquisition order across the entire application, for instance always locking rows in ascending order of their identifier, along with keeping transactions short, avoiding external network calls while holding a database lock, using the least restrictive isolation level the correctness requirement actually needs, and retrying safely where retrying is genuinely appropriate.

Architect principle. The database's deadlock detection is a safety net. The architecture's job is to minimize the probability that a deadlock can occur at all.

A.6 Idempotency

Question. A client sends `POST /orders` with an `Idempotency-Key: abc-123` header. The server creates the order, but the network fails before the client receives the response. The client retries the same request. What should happen?

Developer answer. Create another order.

Architect design. That answer is clearly wrong for anything involving orders or payments. The correct design persists a record containing the idempotency key, a hash of the original request, the resulting order ID, its processing status, and the original response, under a unique constraint on the idempotency key itself. The first request creates this idempotency record and the order together, atomically, within one transaction, and commits. A retry with the same key finds the existing record and returns the original result rather than executing the operation again.

Architect question. Why not store this in Redis instead of the primary database? Because this is correctness-critical state, and keeping the idempotency record and the order itself in the same database allows both to be created within a single atomic transaction, a guarantee an external cache cannot provide as directly.

Architect principle. Correctness-critical state belongs in the system of record unless there is a very deliberate, well-justified reason to place it elsewhere.

A.7 Read replicas

Question. The Orders API receives ten thousand reads per second and five hundred writes per second, and the primary database's CPU utilization is at eighty-five percent. Would you add a read replica?

Developer answer. Yes.

Architect thinking. Diagnose first. Establish specifically what is consuming that CPU, whether it is a poorly written query, a missing index, connection-pool contention, an inefficient ORM-generated query, large unbounded scans, lock contention, or genuinely CPU-heavy aggregation work. Only once the actual bottleneck is identified should the response follow, in roughly this order of preference: optimize the offending query, add the appropriate index, tune the connection pool, and only then consider a read replica, a cache, or further vertical or horizontal scaling of the database itself.

Architect principle. Scale the actual bottleneck. Do not scale the symptom.

A.8 Read replica consistency

Question. A user creates an order, and immediately afterward issues `GET /orders/123`, which is routed to a read replica that has not yet replayed the corresponding WAL. What happens?

Architect answer. The user may see a `404 Not Found`, or a stale prior state of the order. The architectural question this raises is whether this specific read genuinely requires read-after-write consistency. The available strategies are routing reads immediately following a write to the primary instead of a replica, using session or request affinity to pin a user's subsequent reads to the primary for a bounded window, tracking the write's actual replication position (its LSN) and only serving the read from a replica confirmed to have replayed at least that far, or explicitly accepting eventual consistency for read paths where brief staleness genuinely does not matter.

Architect principle. A read replica is not merely a performance feature. It changes the system's consistency characteristics, and that change has to be deliberately accounted for on a per-read-path basis, not assumed away.

A.9 Cascading replication

Question. The topology has grown to one primary with fifty direct replicas, and the primary is showing increasing WAL and network pressure purely from serving that many replication connections. What could you do?

Developer answer. Provision a bigger primary.

Architect thinking. Consider restructuring the topology itself: a small number of replicas connect directly to the primary, and each of those, in turn, streams WAL onward to its own further downstream replicas, reducing the primary's own direct fan-out. This reduces load on the primary, but it introduces a new dependency: a downstream replica's freshness now depends on the health of its specific intermediate upstream replica, and the failure of that intermediate node can affect every replica cascading from it, even while the primary itself, and every other branch of the topology, remains completely healthy.

Architect principle. Scaling a topology very often relocates the bottleneck rather than eliminating it, and simultaneously introduces new failure dependencies that did not exist in the simpler topology.

A.10 Caching

Question. A restaurant menu API receives twenty thousand requests per second, and the underlying menu data changes only a few times per hour. Would you add Redis in front of it?

Developer answer. Yes.

Architect thinking. First establish whether the underlying data is genuinely expensive to compute, how frequently it is actually read, whether some staleness is acceptable to the business, what the database's own current capacity actually is, what cache hit ratio is realistically achievable, and, critically, what happens to the system if the cache itself becomes unavailable. For Tadka specifically, the shape is a straightforward cache-aside pattern: a request checks the cache first, and on a miss, falls through to PostgreSQL and populates the cache for subsequent requests.

Architect principle. Introduce a cache only when you can clearly articulate what specific problem it solves, and can equally clearly describe what happens to the system the moment it fails.

A.11 Cache stampede

Question. A menu cache entry expires at exactly 12:00:00, and five thousand requests arrive in that same instant, all missing the cache simultaneously. What happens?

Developer answer. All five thousand requests fall through and hit PostgreSQL at once.

Architect answer. This is a cache stampede, and it converts a routine cache expiry into a sudden, synchronized burst of database load the database was never provisioned to absorb all at once. The available mitigations include a single-flight lock, where only the first request to miss actually queries the database while the rest wait for, or briefly receive stale data instead of, the freshly computed result; TTL jitter, so keys populated around the same original moment do not all expire at precisely the same instant; stale-while-revalidate serving; and background refresh strategies that update the cache proactively before it ever fully expires.

Architect trade-off. Do not reach for a distributed lock automatically. For a comparatively simple cache and topology, an in-process single-flight mechanism may already be entirely sufficient, and a distributed lock adds coordination complexity that should be justified by the actual topology, not assumed necessary by default.

Architect principle. The real architectural concern is preventing a cache-layer failure from becoming a database-layer failure, not merely reducing the frequency of cache misses.

A.12 Cache invalidation

Question. A restaurant's menu is cached, and the restaurant then updates that menu. What should happen to the cached copy?

Options. Rely on TTL expiry alone, update the cache directly on every write, delete the cache entry on every write and let the next read repopulate it, or drive invalidation from an event stream of database changes.

Architect decision. For a teaching-grade Tadka implementation, the right combination is delete-on-write paired with a TTL as a safety net, because stale menu data should not persist indefinitely, and the explicit delete keeps staleness bounded to "until the very next read" rather than "until the TTL eventually expires."

Architect principle. A caching strategy must explicitly define its freshness guarantee. Deciding only where data is stored, without also deciding how quickly it becomes correct again after a change, is an incomplete design.

A.13 Sharding

Question. The orders table has grown to five billion rows, and a single PostgreSQL instance is becoming difficult to scale further. How would you shard it?

Options. Range-based sharding, hash-based sharding, tenant or customer-based sharding, geographic sharding, or time-based sharding.

Architect thinking. Suppose `hash(customer_id) % N` is chosen. The immediate architectural question is what happens when `N` changes, since a naive modulo scheme ties every key's shard assignment directly to the current shard count, and adding even one additional shard under this scheme forces a massive redistribution of nearly all existing data. The alternatives that avoid this are consistent hashing, an explicit shard-mapping table that can be updated independently of the hash function itself, and virtual shards that can be reassigned to physical nodes without rehashing the underlying keys.

Architect principle. A shard key is not only a decision about how to distribute today's data. It is a decision about how the system is able to evolve tomorrow, and a scheme that makes growth expensive is a real, ongoing architectural cost, not a one-time inconvenience.

A.14 Hot shards

Question. Sharding is applied by `customer_id`, and the overall distribution looks statistically even. One customer, however, is a large enterprise account generating thirty percent of total traffic. What happens?

Developer answer. The hash function should distribute that load evenly across shards.

Architect thinking. It will not, and it is not supposed to: a single customer's identifier hashes to exactly one shard, regardless of how much traffic that one customer generates, so that one shard now carries a disproportionate share of the real workload even while the hash function itself is behaving correctly. The key insight to internalize explicitly is that uniform key distribution is not the same guarantee as uniform workload distribution. Reasonable responses include dedicated partitioning or infrastructure for known large tenants, further sub-sharding a single large tenant's own data, and workload isolation that prevents one tenant's load from degrading service for every other tenant sharing the same shard.

Architect principle. Capacity planning has to account for actual traffic distribution across keys, not merely for how evenly a hash function distributes the keys themselves.

A.15 Payment timeout

Question. A payment gateway normally responds in two hundred milliseconds, the API's own SLA is five hundred milliseconds, and the gateway occasionally takes ten seconds to respond. What should the timeout be set to?

Developer answer. Ten seconds, so the gateway is never cut off before it finishes.

Architect thinking. That setting allows a single slow dependency to occupy the caller's own resources for the full ten seconds on every occurrence, which is exactly backwards from what a timeout is for. Instead, bound the wait to roughly two seconds, derived from the caller's own latency budget rather than the gateway's occasional worst case, and move the payment call off the synchronous request path into asynchronous processing so the customer-facing request is not held open for that duration at all. This immediately raises a further question: what if the payment actually succeeded at the gateway, but the timeout fired before the response made it back? That is an unknown outcome, not a confirmed failure.

Architect principle. A timeout protects your own system from waiting indefinitely. It does not, and cannot, tell you whether the external operation it was waiting on actually succeeded.

A.16 Payment unknown outcome

Question. A charge for one thousand rupees is sent to the gateway, the gateway processes it, but the HTTP request times out before the response arrives, so it is unknown whether the customer was actually charged. Should the request simply be retried?

Developer answer. Yes, retry it.

Architect answer. A naive retry here is genuinely dangerous: if the original charge did succeed and the response was merely lost in transit, a blind retry can produce a second, real charge, resulting in the customer being billed two thousand rupees for a single order. The correct approach combines an idempotency key with the retried request, an explicit payment state machine (moving through states such as created, processing, succeeded, failed, and, importantly, an unknown state pending reconciliation), and a reconciliation process that eventually resolves any charge left in that unknown state by directly querying the gateway for its authoritative outcome.

Architect principle. In a distributed system, "we do not yet know the outcome" is a legitimate, first-class state that the design must explicitly represent, not an error condition to be papered over with an automatic retry.

A.17 Bulkhead

Question. A payment gateway becomes slow, and all one hundred of the application's worker threads end up waiting on payment calls simultaneously. What happens to the rest of the application?

Answer. Payment has consumed the application's entire concurrency budget, so every other operation that also needs a worker thread, capabilities that have nothing to do with payment, is starved of capacity as a side effect. A bulkhead prevents this by giving payment its own dedicated, bounded concurrency limit, for instance ten workers rather than access to the shared pool of one hundred, so that other workloads retain their own capacity regardless of how badly payment is currently behaving.

Architect principle. Isolation is not only achievable by running components in separate processes. Resources can, and often should, be isolated from one another even within a single process.

A.18 Circuit breaker

Question. A payment gateway is currently returning errors for one hundred percent of requests. Should the application keep sending requests to it?

Answer. No. A circuit breaker tracks the failure rate, transitions from a closed (normal) state to an open state once a failure threshold is crossed, rejects requests immediately without even attempting the call while open, and after a defined recovery window transitions to a half-open state that sends a small number of probe requests to test whether the dependency has recovered, closing again on success or reopening on continued failure.

Architect question. What does a circuit breaker actually solve? It does not fix the payment gateway, and it cannot. It prevents the calling system from continuing to spend its own resources on calls to a dependency that is already known, with high confidence, to be failing.

Architect principle. Resilience patterns protect your own system from a dependency's bad behavior. They do not, and are not intended to, make the dependency itself more reliable.

A.19 Asynchronous processing

Question. A customer creates an order, and the payment step takes two seconds to complete. Should POST /orders wait for those two seconds before responding?

Answer. Not necessarily. The order can be created immediately, marked with a `PaymentPending` status, and returned to the customer right away, with the actual payment processed in the background afterward. This immediately raises the follow-up question of where that background job is actually stored while it waits to be processed. If it exists only in the application process's own memory, a process crash or restart before the job is processed causes it to disappear entirely, which is precisely the durability wound this document deliberately exposed in Module 6.

Architect principle. Asynchronous processing removes temporal coupling from the customer-facing request, but in doing so it introduces a durability requirement for the deferred work that a synchronous design never had to solve, because the work never had to survive on its own between two points in time.

A.20 Service extraction

Question. Payment currently lives inside the monolith. Should it be extracted into its own service?

Weak developer answer. Yes, because the industry is moving toward microservices.

Architect answer. Ask, explicitly and in this order: what specific problem are we solving; does that problem actually exist today, with evidence, rather than as a hypothetical; can the monolith solve it, and if so, at what cost; and what new failure modes does extraction itself introduce. The legitimate drivers worth weighing are fault isolation, a security or compliance boundary, data ownership, independent scaling, independent deployment, and distinct team ownership. Whatever the answer, the cost of extraction has to be calculated explicitly too: a new network hop, a separate deployment pipeline, separate monitoring, distributed debugging, dedicated failure handling, a separate database, and the consistency complexity that a separate database necessarily introduces.

Architect principle. A service boundary is a trade, exchanging some simplicity for some isolation. It is not an unconditional achievement to be pursued for its own sake.

A.21 Database-per-service

Question. Payment has been extracted into its own service. Should it continue using the same database Order Service already uses?

Developer answer. Yes, because it is easier to set up that way.

Architect answer. Sharing a database between two independently deployed services produces separate services with shared, un-owned data, since nothing structurally prevents Order Service from reading or writing Payment's tables directly, bypassing every validation rule and audit trail Payment Service's own code exists to enforce. The correct topology gives each service its own database: Order Service owns Order DB, Payment Service owns Payment DB, and Payment now genuinely owns its own state. The cost of this correctness is real: local, single-database transactions across the two concerns are no longer possible, and the system now has to solve for distributed consistency explicitly instead of getting it for free from a shared transaction.

Architect principle. Service ownership without a corresponding data-ownership boundary is, in practice, usually a weak boundary that only looks strong on a diagram.

A.22 HTTP versus a message broker

Question. Order Service needs to inform Payment Service that a payment should be attempted. Should this communication go through Kafka?

Developer answer. Yes, because microservices are supposed to communicate asynchronously.

Architect answer. Not automatically, and not by default. For the specific requirement of Order telling Payment "please attempt this charge," a direct HTTP call may be entirely sufficient today. A message broker becomes genuinely attractive once there is an actual, evidenced need for durable asynchronous messaging, high sustained throughput, multiple independent consumers of the same event, message replay, deliberate decoupling between producer and consumer lifecycles, or a broader event-driven workflow that spans more than one direct producer-consumer relationship.

Architect principle. Choose infrastructure because a concrete requirement demands it, never because a particular architecture diagram looks more modern with that infrastructure included in it.

A.23 Enumerating failure across a network boundary

Question. Order Service calls Payment Service over HTTP. What can actually go wrong with that single arrow on the architecture diagram?

Expected enumeration. DNS resolution failure, connection refused, connection timeout, request timeout while waiting for a response, an HTTP 500 server error, an HTTP 429 rate-limit rejection, a malformed or truncated response body, a network partition separating the two services entirely, the Payment process crashing mid-request, the Payment database becoming unavailable while Payment's own process is still up, and the external payment gateway itself becoming unavailable one layer further downstream.

The architect question that follows each one. For every failure on that list, what does Order Service specifically do about it? This is the concrete difference between drawing a service boundary on a diagram and actually operating one in production: a diagram can show the arrow; only an explicit failure-handling design determines whether the arrow behaves safely when it breaks.

A.24 The large-scale Tadka system design scenario

Question. Design Tadka's order and payment system to support ten million orders per day, with peak traffic running at twenty times the average rate, a payment gateway that can become slow or fully unavailable at any time, a hard requirement that customers must never be double-charged, an expectation that order creation remains responsive even under stress, menu reads that vastly outnumber menu writes, an order-history read pattern that is heavily read-biased, a requirement for a clear ownership boundary around payment data specifically, and an overall requirement that the system survive the failure of any individual service without a total outage.

What a complete design has to address. The public API surface, the database and its indexing strategy, pagination, the caching layer, read replicas, the replication topology underlying them, sharding if and when data volume demands it, the payment integration and its asynchronous processing design, idempotency, the full set of resilience patterns from timeout through circuit breaker, the service boundaries actually chosen and their justification, and the observability needed to know, in production, whether all of the above is actually working as designed.

The discipline this scenario is meant to teach. A design is not complete once the diagram is drawn. It becomes real only once it has been actively attacked, which is the exercise Appendix B walks through in full.

Appendix B: The Architect Defense Round

Appendix A tested individual topics in isolation. This appendix takes the complete system design from Question A.24 and subjects it to a sequence of deliberate attacks, the same discipline Module 5 applied specifically to the Payment extraction, now applied to the whole system at once. Each attack below should be treated as a live question to answer out loud, in specific, falsifiable terms, not as a rhetorical prompt.

Attack 1: "Redis is down." The expected first-order answer is that every cache miss falls through to PostgreSQL. The follow-up that actually tests the design is: what if Redis is down for thirty minutes, not thirty seconds? At that duration, the question is no longer about correctness, it is about whether the database has the provisioned capacity to sustain the full, un-cached read load for the entire outage window, a capacity question that has to be answered with a number, not a shrug.

Attack 2: "The primary database is down." Can reads continue, and from where? Can writes continue at all, given that PostgreSQL's standard topology has exactly one writable primary? How, specifically, does a replica get promoted, and what mechanism prevents the old primary from causing split-brain if it comes back online unexpectedly? What is the actual, current replication lag at the moment of failure, and therefore how much recently-committed data might be missing from the newly promoted primary?

Attack 3: "The Payment service is down." The expected answer, if the design is sound, is that orders continue to be created in a `PaymentPending` state. The attack does not stop there: where, specifically, is that pending payment job stored while Payment Service is unavailable? If the honest answer is "in memory," the design has just rediscovered Module 6's durability wound in front of the interviewer, live, which is itself a legitimate and valuable answer if named explicitly as a known, deliberate gap rather than glossed over.

Attack 4: "The payment succeeded, but the service crashed before recording that success." The expected concepts in the answer are idempotency (so a retry of the same logical payment does not double-charge), durable state (so the fact that a charge was attempted is not lost along with the crashed process), and reconciliation (an explicit, periodic process that resolves any payment left in an ambiguous state by checking the gateway's own authoritative record).

Attack 5: "Traffic increases tenfold." The unacceptable answer is "add more compute instances." The acceptable answer identifies, with a specific number, which component saturates first under that tenfold increase, and why that specific component is the one that fails before any other, rather than treating every component as scaling identically and simultaneously.

Attack 6: "Redis, Payment, and the message broker are all down simultaneously." This is a graceful-degradation question: what functionality can, and should, remain available when every non-essential dependency has failed at once? A well-designed system should be able to state precisely which capabilities survive (likely: viewing the menu from a database fallback, viewing existing order history) and which are explicitly, acceptably degraded (new payment processing, delayed until dependencies recover).

Attack 7: "Infrastructure cost must decrease by forty percent." This question has no purely technical answer, which is exactly what makes it valuable: it forces an explicit trade-off conversation. Reasonable candidates include removing replicas that were provisioned ahead of actual, evidenced need, reducing general overprovisioning, deferring a message broker that was adopted before its requirement genuinely existed, reducing cache footprint, adjusting data retention policies, or restructuring how managed services are used. Every one of those reductions has a consequence, and

naming the consequence honestly, not just the saving, is what the answer is actually being evaluated on.

Appendix C: Configuration and Cross-Stack Reference

C.1 Setting up asynchronous streaming replication

On the primary (postgresql.conf):

```
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
```

Create a dedicated replication role and a replication slot for the standby:

```
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '...';

SELECT * FROM pg_create_physical_replication_slot('replica1_slot');
```

Allow the replication connection (pg_hba.conf on the primary):

host	replication	replicator	10.0.0.0/24	scram-sha-256
------	-------------	------------	-------------	---------------

Take a base backup and configure the standby (run on the replica host):

```
pg_basebackup -h primary-host -D /var/lib/postgresql/data \
-U replicator -P --wal-method=stream
```

On the standby, enable standby mode and point it at the primary (postgresql.conf, PostgreSQL 12 and later):

```
primary_conninfo = 'host=primary-host port=5432 user=replicator password=...
application_name=replica1'
primary_slot_name = 'replica1_slot'
```

```
touch /var/lib/postgresql/data/standby.signal
```

C.2 Synchronous replication

On the primary (postgresql.conf):

```
synchronous_standby_names = 'ANY 1 (replica1, replica2)'
synchronous_commit = on
```

This configuration commits a transaction only once at least one of replica1 or replica2 has flushed the corresponding WAL to durable storage, using the quorum syntax so that either standby being temporarily unavailable does not, by itself, block every write on the primary.

C.3 Cascading replication

On the intermediate standby, in addition to its own standby configuration, allow it to accept further downstream replication connections the same way the primary does:

```
max_wal_senders = 10
max_replication_slots = 10
```

On the downstream standby, point `primary_conninfo` at the intermediate node, not at the original primary:

```
primary_conninfo = 'host=replica1-host port=5432 user=replicator password=...'
```

C.4 Monitoring replication lag

```
SELECT
    client_addr,
    application_name,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    replay_lag
FROM pg_stat_replication;
```

`replay_lag` (available since PostgreSQL 10) reports the actual time delay, not just the LSN distance, between a transaction committing on the primary and it becoming visible to queries on the standby, which is the number that should back any alert on replication lag.

C.5 Read/write connection routing per stack

.NET, using two named connection strings and an explicit routing decision in the repository layer:

```
public sealed class OrderReadRepository
{
    private readonly NpgsqlDataSource _replicaDataSource;

    public async Task<Order?> GetByIdAsync(long orderId, bool requireFreshRead)
    {
        var dataSource = requireFreshRead ? _primaryDataSource : _replicaDataSource;
        await using var conn = await dataSource.OpenConnectionAsync();
        // query against the selected connection
    }
}
```

Java, with a routing `DataSource` that picks primary or replica based on a thread-local hint set by the calling code:

```
public class ReplicaAwareRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        return ReadPreferenceContext.isReadAfterWrite() ? "primary" : "replica";
    }
}
```

Node.js, with a thin wrapper choosing the pool per query:

```
function query(sql, params, { requireFreshRead = false } = {}) {
    const pool = requireFreshRead ? primaryPool : replicaPool;
    return pool.query(sql, params);
}
```

Go, with an explicit interface selecting the underlying *sql.DB:

```
func (r *OrderRepository) db(requireFreshRead bool) *sql.DB {
    if requireFreshRead {
        return r.primary
    }
    return r.replica
}
```

Every one of these examples encodes the same architectural rule from Section 2.3: the routing decision is explicit, per call, based on whether that specific read needs the primary's guarantees, never a blanket, connection-string-level choice made once for an entire service.

C.6 Idempotency key and durable payment job schemas

```
CREATE TABLE payment_idempotency_keys (
    idempotency_key TEXT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    request_hash TEXT NOT NULL,
    status TEXT NOT NULL, -- 'PENDING' | 'SUCCEEDED' | 'FAILED'
    gateway_response JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE payment_jobs (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(id),
    status TEXT NOT NULL DEFAULT 'PENDING',
    attempts INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    next_attempt_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Creating the order and its durable payment job atomically (Section 6.3):

```
BEGIN;

INSERT INTO orders (id, status, ...) VALUES ($1, 'Created', ...);

INSERT INTO payment_jobs (order_id) VALUES ($1);

COMMIT;
```


Appendix D: Glossary

B-tree index locality: how close together, in physical index storage, keys that are inserted around the same time end up; poor locality (as with random UUIDs) causes more page splits and slower inserts as a table grows.

Base backup: a full, physical copy of a PostgreSQL data directory, typically taken with `pg_basebackup`, used as the starting point for a new standby or for point-in-time recovery.

Bulkhead: a control limiting how much concurrent capacity a specific dependency may consume, preventing it from starving every other capability sharing the same process.

Cascading replication: a topology in which a standby simultaneously receives WAL from an upstream node and forwards it to its own downstream standbys, reducing the primary's direct connection fan-out at the cost of a new dependency chain.

Checkpoint: a periodic PostgreSQL operation guaranteeing all data-page changes up to a given WAL position are durably on disk, bounding how much WAL crash recovery must replay.

Circuit breaker: a control that stops calling a dependency altogether once it is confirmed to be failing, to avoid wasting resources on calls unlikely to succeed.

Cluster (sharding sense): see Sharding; not to be confused with a PostgreSQL "cluster," which refers to a single server instance and all the databases it manages.

Consistent hashing: a key-to-shard assignment scheme designed so that changing the number of shards moves only a small, bounded fraction of existing keys, unlike a naive modulo scheme.

Deadlock: a cycle of transactions each waiting on a lock the other holds, detected and resolved by the database aborting one of the participating transactions.

Fault isolation: the property that a failure in one component cannot directly cause a failure in another, most strongly achieved through separate operating-system processes.

Fencing: forcibly ensuring a previously failed or displaced primary can no longer accept writes, a mandatory part of any safe failover mechanism, preventing split-brain.

Hot shard: a single shard receiving disproportionate workload relative to its peers, despite an otherwise well-distributed hash function, because one key (such as one large customer) legitimately carries a large fraction of real traffic.

Hot standby: a PostgreSQL standby configured to serve read-only queries while simultaneously replaying incoming WAL.

Idempotency key: a stable identifier attached to an operation so that a repeated attempt (a retry) can be recognized and safely deduplicated rather than re-executed.

Keyset (cursor) pagination: a pagination technique that fetches rows relative to the last-seen row's sort key, avoiding the linear cost of skipping rows that offset-based pagination incurs at deep pages.

Logical replication: WAL decoded into row-level logical changes and applied via publications and subscriptions, supporting selective, cross-version replication, unlike physical replication.

Log Sequence Number (LSN): a monotonically increasing address identifying a position in the WAL stream, the unit every replication and recovery mechanism uses to measure progress.

MVCC (Multi-Version Concurrency Control): PostgreSQL's concurrency model, giving readers a consistent snapshot without blocking writers, at the cost of needing explicit handling for lost updates.

Optimistic locking: a concurrency-control strategy that makes a write conditional on a row's version being unchanged since it was read, detecting conflicts rather than preventing them upfront.

Pessimistic locking: a concurrency-control strategy that acquires an explicit row lock at read time, blocking other transactions from the row until the current one finishes.

Physical replication: streaming the raw WAL to produce a byte-for-byte copy of the primary's data pages, cluster-wide.

Point-in-time recovery (PITR): restoring a base backup and replaying archived WAL forward to a specific target point, used to recover from a mistake such as an accidental data deletion.

Primary: the PostgreSQL instance accepting writes and generating the WAL stream for a replication topology.

Read-after-write consistency: the guarantee that a read immediately following a write by the same actor reflects that write, not automatically provided once reads are served from a replica.

Replication lag: the gap between the primary's current WAL position and a replica's most recently replayed position, an inherent property of asynchronous replication.

Replication slot: a named, persistent marker on the primary retaining WAL until a specific consumer has consumed it, preventing premature deletion at the cost of potential unbounded WAL growth if that consumer disappears.

Replica (standby): a PostgreSQL instance receiving and replaying another instance's WAL stream, optionally serving read-only queries.

Sharding: splitting a dataset across multiple independent database instances (or partitions) by a chosen key, to scale beyond what a single instance can hold or serve.

Split-brain: two nodes simultaneously believing themselves to be the sole primary and independently accepting writes, typically caused by a failover that promoted a standby without fencing the original primary.

Streaming replication: PostgreSQL's mechanism for continuously shipping WAL from a primary's WAL sender process to a standby's WAL receiver process.

Synchronous replication: a replication mode in which the primary waits for acknowledgment from one or more standbys before confirming a commit to the client, trading write latency for a stronger durability guarantee.

Write-Ahead Log (WAL): PostgreSQL's durability mechanism, requiring every change to be durably logged before the corresponding data-page change is considered complete.

Appendix E: Further Reading

- The official PostgreSQL documentation's chapters on Write-Ahead Logging, Reliability and the Write-Ahead Log, Log-Shipping Standby Servers, Streaming Replication, Replication Slots, Logical Replication, and Continuous Archiving and Point-in-Time Recovery: the primary, authoritative source for exact current configuration parameters and defaults, which should always be checked against the specific PostgreSQL version in use.
- Patroni's own documentation: the standard reference for automated PostgreSQL failover, quorum-based leader election, and the fencing mechanisms that prevent split-brain during a failover.
- Martin Kleppmann, *Designing Data-Intensive Applications*: the standard reference for replication, partitioning, and consistency concepts underlying Modules 1, 2, and 9 of this document.
- Vladimir Kuznetsov and other PostgreSQL performance writers on UUID versus sequential key index locality, and the PostgreSQL Wiki's coverage of UUIDv7 and time-ordered identifiers: useful further depth on Section 7.3's identifier-design trade-off.
- Use The Index, Luke (an independent, widely used SQL indexing reference): a practical, engine-agnostic companion to Module 7's indexing and pagination discussion.
- Martin Fowler and Chris Richardson on service decomposition and the database-per-service pattern: further depth on Module 3 and Module 4's extraction and ownership reasoning.
- Google's Site Reliability Engineering book and its companion Site Reliability Workbook: the standard reference for the service-level-objective and failure-analysis thinking underlying Module 10's meta-framework.